

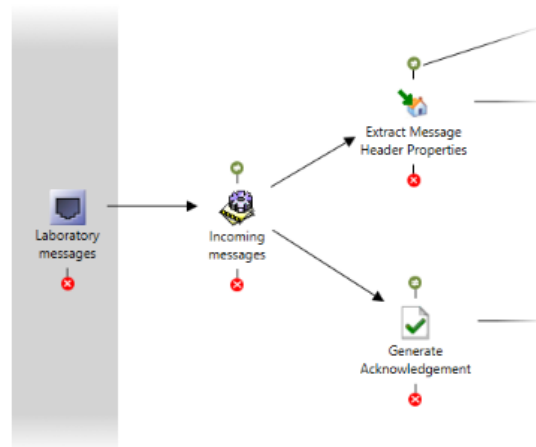
Communication Point Modes

Each communication point can be configured to run in one or more modes. The modes supported vary by communication point type and depend on the behavior of the interface it implements. The full set of supported modes are:

- **Input**
Accepts inbound messages from one or more external systems.
- **Output**
Transmits outbound messages to one or more external systems.
- **In->Out**
Synchronous; accepts an input message and then prevents further input until a message has been sent.
- **Out->In**
Synchronous; transmits a message and then prevents further output until a response has been received.
- **Bidirectional**
Asynchronous; accepts input messages and transmits outbound messages.

When a communication point is configured to use a mode that can both send and receive messages, it will appear on the input and output sides of a route.

It is recommended that a single connection is configured from each Input communication point. This ensures that processing to parse messages and extract properties happens only once. This helps minimize the processing required when the message is loaded into the engine. Where messages are used on multiple routes, a single input makes it easier to track the source of messages received and processed by Rhapsody.



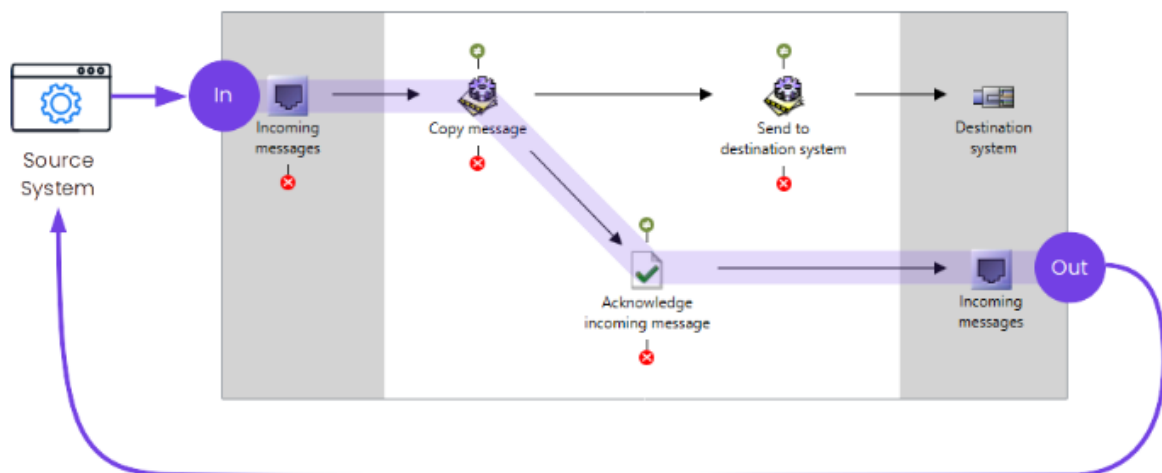
By convention, the input and output for a communication point will appear on the same route. When building a solution this helps you visualize the data flow to and from the external system.

Be careful when configuring a communication point in one of the two synchronous modes (In->Out and Out->In), as it is possible for source and destination systems to get

out of synchronization with Rhapsody. In the next lesson we will discuss the configuration of these modes.

In->Out Mode

In In->Out mode, messages are received from a communication point. Any modifications are made to the message, then a message (typically an acknowledgement message) is sent back to the originating communication point before the next message is received.



In In->Out mode:

- The communication point starts in Input mode and will not send any messages until a message is received.
- When a message is received, the communication point switches to Output mode and will not accept additional input messages.
- The communication point remains in Output mode until either a message has been sent or the response timeout period has elapsed, then it goes back to Input mode.
- Rhapsody will only process a single message received by a communication point at a time.

When operating a communication point in In->Out mode, be sure every received message generates an output message to the same communication point.

- If the number of messages received by the communication point does not match the number of messages sent to the communication point, it is possible for some messages to be delayed.
- When the number of messages received exceeds the number sent, receipt of additional messages will be delayed.

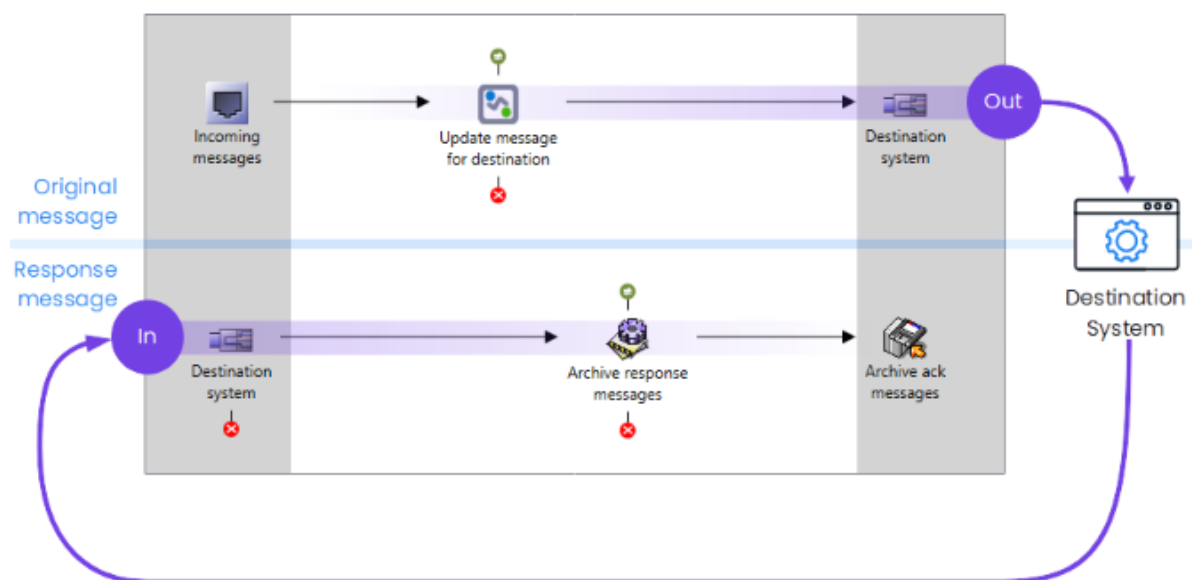
- When more messages are sent than received, there will be a queue of messages in Rhapsody waiting to be sent.

You should try to ensure that message processing within Rhapsody does not take an excessively long time, as this will delay the response message and the processing of subsequent messages. Where synchronous receipt and processing of messages is not required, Bidirectional mode should be used in preference to In->Out mode.

Out->In Mode

In Out->In mode, a message is sent to a destination system. Rhapsody then waits for the response before sending the next message.

When a route is created that uses a communication point in this mode, the path for the original message and the response will frequently not be connected to each other. This mode is typically used when the external system cannot safely process more than one concurrent message.



When a communication point operates in Out->In mode:

- Rhapsody assumes that a message received on a connection is a response to the preceding output message on the same connection.
- The message properties associated with the outbound message will be copied to the response input message. This makes it easier to perform actions on the response message that rely on properties associated with the original message.

Bidirectional Mode

Bidirectional mode can be used to receive messages into Rhapsody (similar to In->Out mode) or send messages to another system (similar to Out->In mode). This

asynchronous nature means that Rhapsody can send and receive multiple messages from the communication point at the same time. When considering this mode for a communication point (especially a TCP-type communication point), you should check that the external system can manage multiple messages being sent and received at the same time.

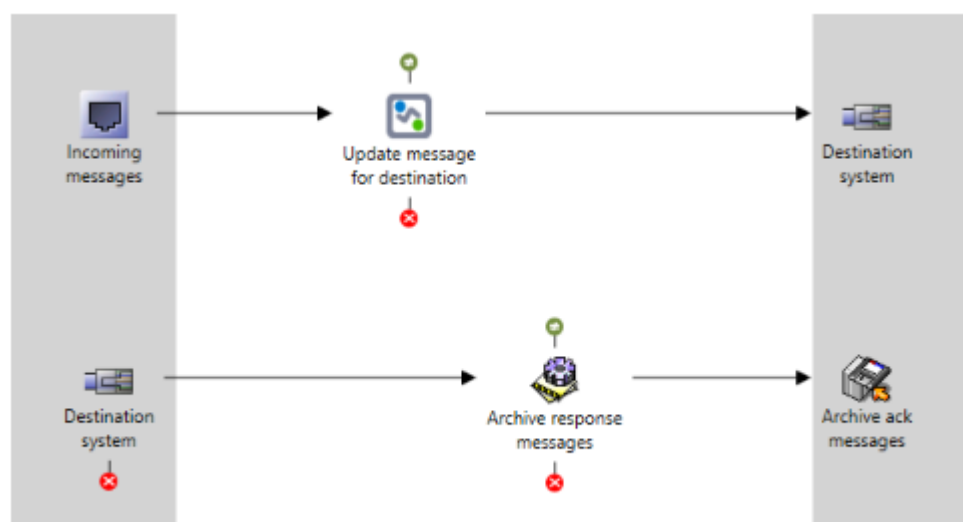
Communication Models

Many systems that send messages to Rhapsody require an acknowledgement in return before they will send another message. The Rhapsody Engine can be configured to support the following communication models on an interface-by-interface basis:

1. 1

Pass Through

The engine acts as arbitrator and manager for the messages, with responses (acknowledgements) from the recipient system passed back to the sending system.

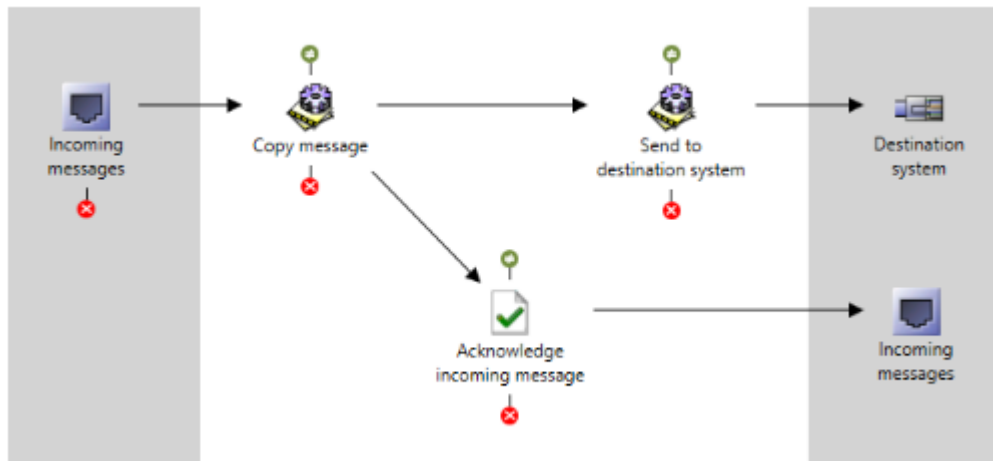


In a pass-through interface, the response to the sending system contains information provided by the recipient system (for example, a register patient message that includes the patient's new identifier in the response). This is important when the sending system uses the response to understand how the recipient system has processed the message. The downside is that it is generally not possible to send a single message to multiple recipient systems and merge all responses into one for the sending system.

1. 2

Store and Forward

The engine accepts responsibility for a message and ensures it is correctly received by the intended recipient.



In a store and forward interface, acknowledgement messages are returned to the sending system before delivery to any destination systems. This allows you to send the message to multiple destination systems. It also improves throughput because acknowledgement is sent to the sending system before delivery of the message. This lets the sending system send subsequent messages sooner.

A store and forward interface will make your Rhapsody configuration responsible for detecting and managing error messages returned from destination systems.

Advanced Routing

With advanced routing, additional tasks can be performed on messages after they have either successfully or unsuccessfully been sent outbound by a communication point.

Continued Routing on Success

Continued Routing on Success can be used with communication points configured in Output mode. It allows further processing on a message that has been sent to an external system without needing a response (normally in the form of an acknowledgement message) from the external system.

This feature is enabled from the communication point's **Connection** tab.

When enabled, an icon is added to the communication point. It can then be added to the input side of the route where successfully delivered messages will be routed for additional processing.

Icon: Continue routing on success.

Continued Routing on Failure

Continued Routing on Failure can be used with a component that is configured in Output, Out->In, and Bidirectional modes. It allows further processing on messages

that have encountered an error while being sent. This could involve sending an error report of the failure to an administrator or sending the message to the external system via an alternate method.

① Connection Routing on Failure cannot be used with the Sink, Dynamic Router, or Error Message Redirector communication points because they send messages to another route in Rhapsody (or, in the case of Sink, delete the message) and can never fail to send.

To use this feature, the communication point is added to the input of a route after the communication point has already been added as an output. An error connector (similar to the error connector on filters) is used to process any connection errors.

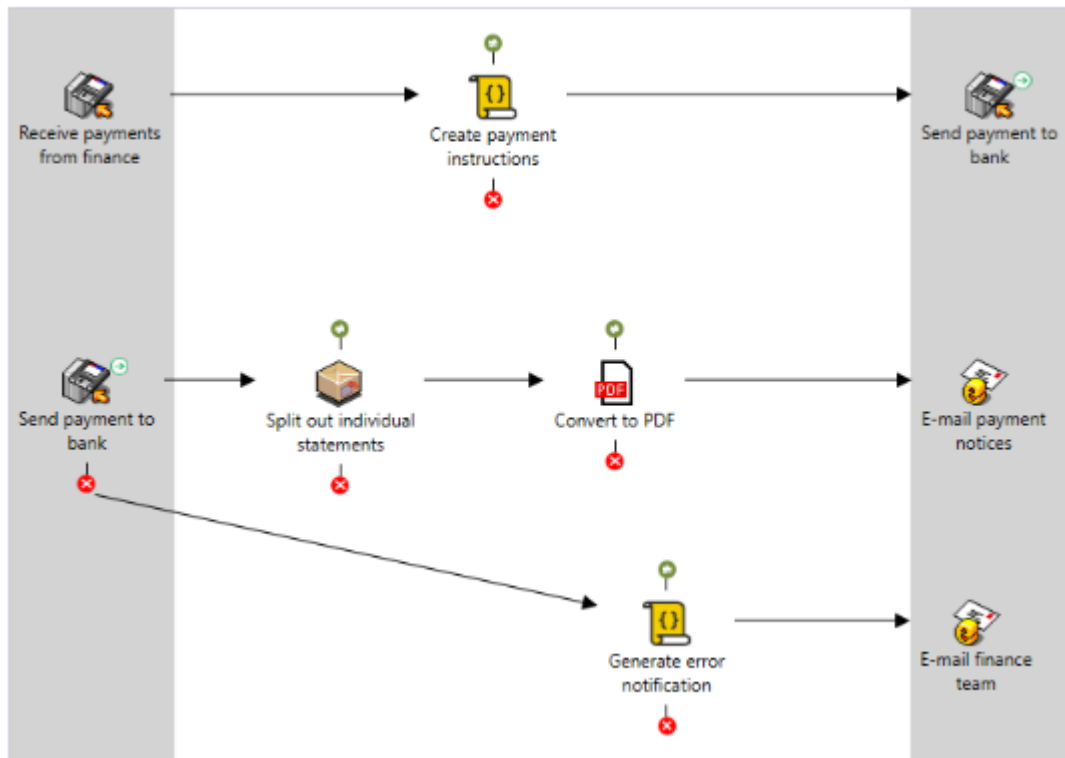
Rhapsody sets the following message properties on all messages to help process failures.

- `rhapsody:ErrorCause`
A description of the failure.
- `rhapsody:ErrorCommunicationPointId`
The internal id for the communication point that generated the failure.
- `rhapsody:ErrorCommunicationPointName`
The name of the communication point that generated the failure.

Example

In the following example, the Send Payment to Bank communication point has been configured in Output mode. Both Continued Routing on Success (the green arrow in the circle) and Continued Routing on Failure (the error connector) have been used.

- On success, a PDF is sent containing the payment notice.
- On failure, an error notification is sent to the finance team.

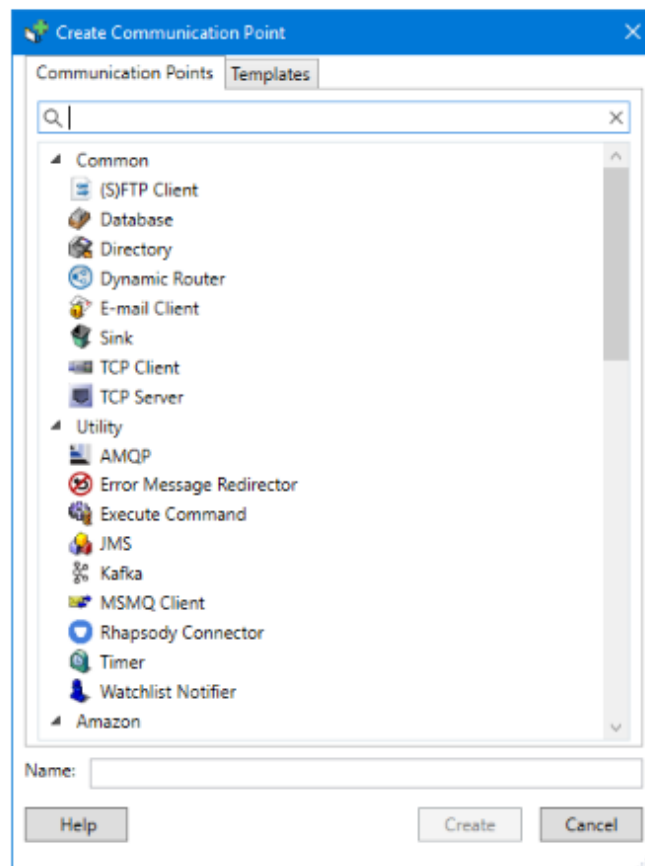


Creating a New Communication Point

You can create a new communication point in one of the following ways:

- Right-click in the **Workspace**, then select **New Communication Point** from the context menu.
- Select **New Communication Point** in the IDE toolbar.
- From the **Insert** menu on the IDE toolbar, select **New Communication Point**.

Each of these options opens the **Create Communication Point** dialog shown below. The most commonly used communication points appear at the top of the list of quick access. We will focus on the Directory and Sink communication points.



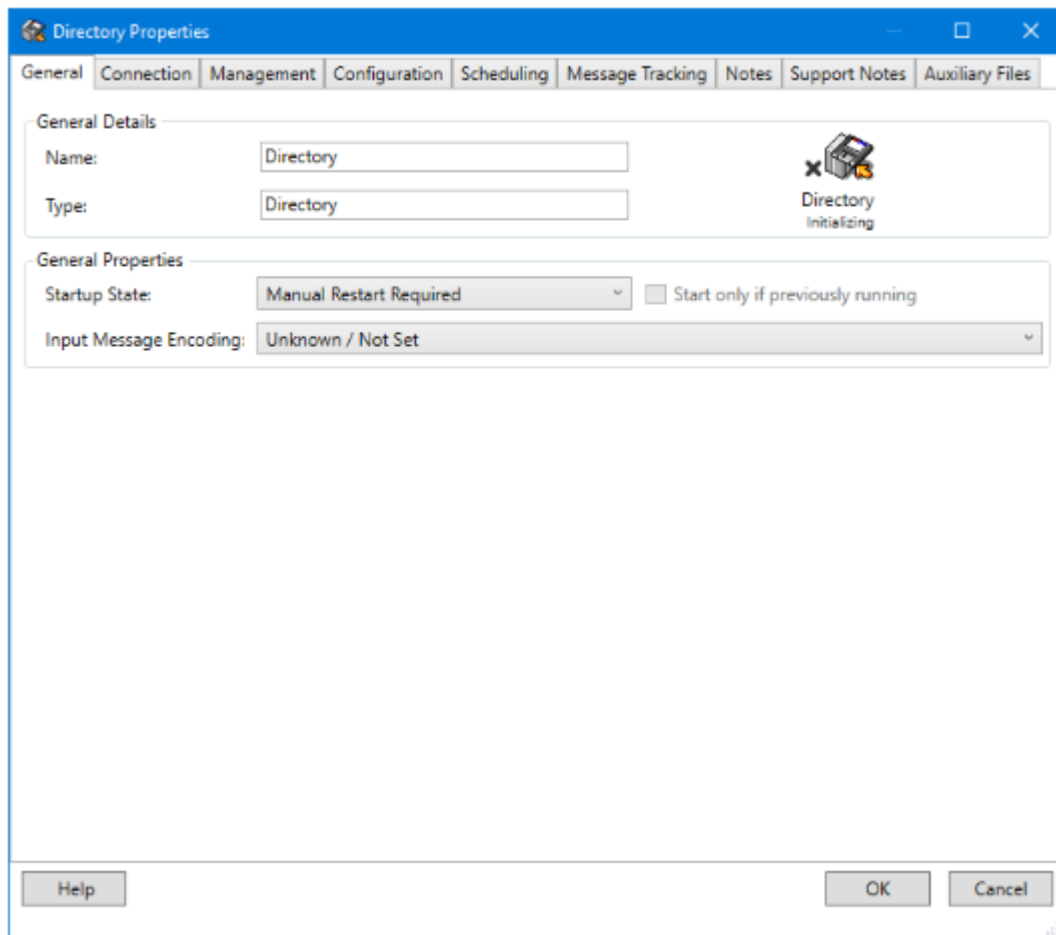
Double-click a communication point's icon, or single-click the icon followed by the **Create** button, to open the new component's **Properties** dialog where you can begin its configuration.

Use the search box to filter the list of available communication points. Use **Templates** to create a pre-configured communication point configuration. The creation of templates is not covered in this course.

The Properties Dialog

A communication point's function and activity are controlled by a set of properties accessed via its **Properties** dialog. To open this dialog, double-click the icon in either the **Workspace** or route editing canvas, or right-click to open the context menu.

The **General** tab opens by default for new communication points. For existing communication points the **Configuration** tab opens by default. The following example shows the **General** tab for a newly created Directory communication point.



All **Properties** dialogs share a similar tab structure to simplify navigation and management.

- **General**
Provides the communication point's name and startup state.
- **Connection**
Controls the operating mode and options relating to how it sends and receives messages.
- **Management**
Applies limits to the volume of incoming and outgoing messages handled by this communication point.
- **Configuration**
Sets this communication point's protocol-specific parameters.
- **Scheduling**
Controls the days and times at which this communication point is actively processing messages.

- **Message Tracking**
Configures automatic matching of messages and their responses based on defined schemes.
- **Notes**
Free text documentation relating to this communication point's purpose and configuration details.
- **Support Notes**
Notes to assist in the support of this communication point.
- **Auxiliary Files**
Lists the drivers and control files specific to the configuration of this communication point.

Common Features

Some configuration options depend on other settings. For example, the **Retries Exceeded Action** option in the **Connection Retries** panel will only be configurable when the **Retries Per Message** property is set to any value other than **Infinite**.

Communication Point Help

You can view information relating to the function and configuration of a communication point by selecting the **Help** button at the bottom of the configuration window.

General Tab

A communication point's name can be modified on the **General** tab. An icon identifies its status as **Started**, **Stopped** or **Initializing** (which is displayed when configuration is not complete, and the component has not been checked in).

General Properties

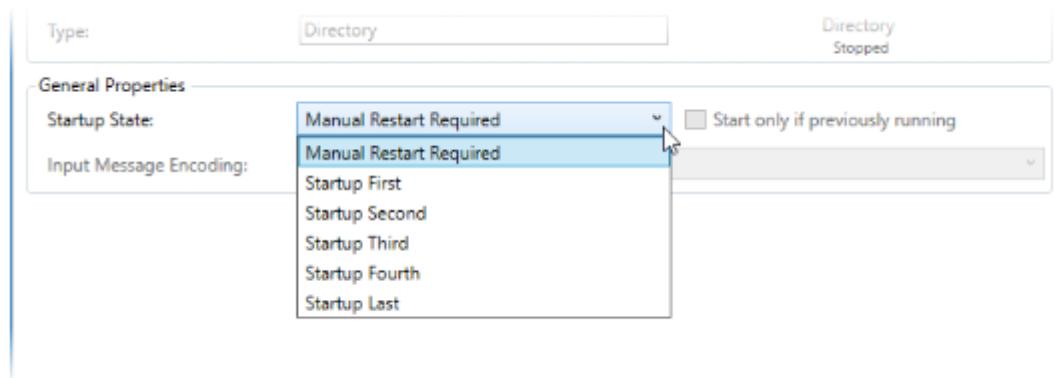
Startup State

Rhapsody can control the order in which components start when the engine starts.

- **Manual Restart Required**
Ensures the communication points do not start automatically.
- **Startup First...Last**
Sets the order in which the component starts.

When the engine is started, Rhapsody attempts to simultaneously start all components with the same startup state at the same time. If a component has been stopped prior to

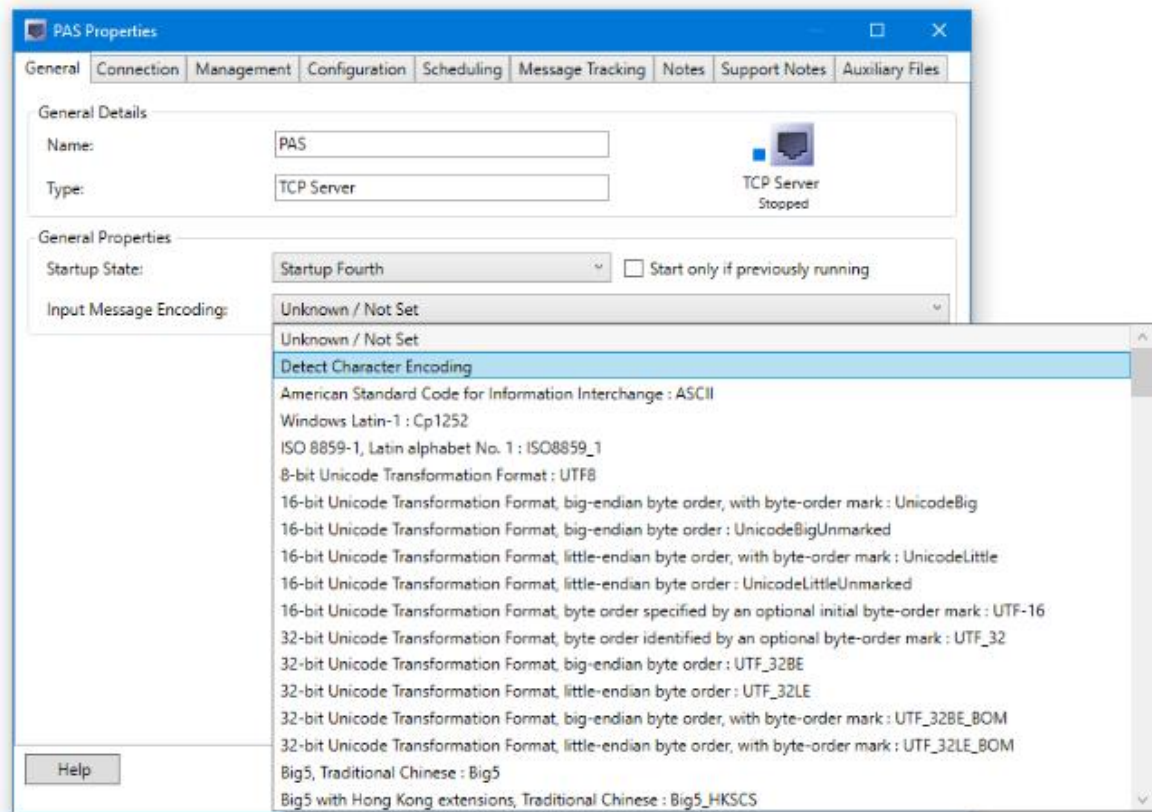
an engine restart and if you do not want it to start automatically, select the **Start only if previously running** checkbox.



Input Message Encoding

Rhapsody maintains a message property which identifies a message's character encoding system. The message encoding can be set on all communication points that can receive messages. The property is used to decode the message when processing the contents of the message body.

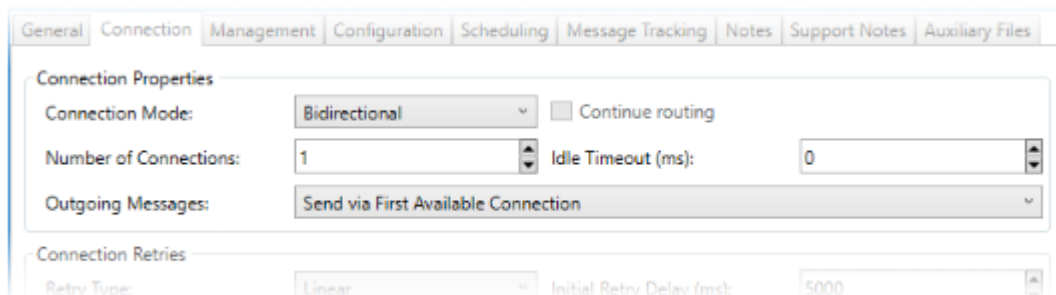
The engine supports character encoding schemes defined by Java. If the default value of **Unknown / Not Set** is selected, Rhapsody uses the language and locale settings of the Operating System to guess the appropriate message encoding. If the encoding set is likely to vary on a message-by-message basis, you can use the **Detect Character Encoding** setting. When using this option, heuristics including byte order marks and character sequences in the message are used to select the most likely encoding.



Connection Tab

The **Connection** tab contains a set of properties that define how the communication point maintains the connection with the external system and what to do if there is a connection fault.

Connection Properties



Connection mode

The **Connection Mode** sets the mode this communication point operates in.

The **Continue routing** checkbox enables a communication point operating in Output mode to re-route a message upon successful message transmission so that further actions can be performed on the message.

Number of connections

Many communication points can maintain an active connection with multiple systems at the same time. The **Number of Connections** property defines how many connections the communication point will make. For some server communication point types, such as TCP Server, this setting determines how many external systems can simultaneously connect. The Directory communication point is an exception. The value of this property should always be **1**.

The **Idle Timeout** is the number of milliseconds of inactivity that must pass before the connection is closed. If left at the default value of **0**, the connection will never timeout.

Outgoing messages

When a communication point is operating in either In->Out or Bidirectional mode, connection tracking ensures a response message generated by a Rhapsody configuration can be sent on the same connection as the original message. This option is particularly important when a communication point has multiple connections and is receiving input from multiple systems.

In->Out mode

For communication points configured as In->Out, Rhapsody always attempts to send the response message to the same connection that the original message was received on. The options in this mode control the behavior of the communication point when the original connection has been closed (either delete or send to Error Queue).

Bidirectional mode

When a communication point has been configured in Bidirectional mode, there are four options. These allow you to configure the communication point to operate as an asynchronous version of either an Out->In or In->Out communication point. The four options are:

- Send via first available connection
- Send via original connection (keep until connection is available)
- Send via original connection (discard if original connection has closed)
- Send via original connection (move to Error Queue if original connection has closed)

① When a Bidirectional communication point is truly asynchronous (input and output are completely independent), or if it is output driven (meaning it is effectively operating in an asynchronous Out->In mode), the **Send via first available connection** setting must be used.

Connection Retries

The **Connection Retry** settings control the way Rhapsody handles the failure of a connection to or from a remote system. They determine whether an attempt should be made to reconnect, as well as the frequency and duration of those attempts. If the connection retries are exceeded, the communication point will be stopped.

Retry Type

The screenshot shows the 'Connection Retries' configuration window. At the top, 'Outgoing Messages' is set to 'Send via First Available Connection'. Below this, the 'Connection Retries' section contains several settings: 'Retry Type' is set to 'Linear', 'Initial Retry Delay (ms)' is 5000, 'Number of Retries' is 5, 'Maximum Retry Delay (ms)' is 300000, 'Retries per Message' is 'Infinite', 'Message Retry Delay (ms)' is 1000, 'Retries Exceeded Action' is 'Shut down communication point', 'Redirect Target' is 'Select Communication Point', and 'Included Messages' is 'All Messages'. At the bottom, 'Out->In and In->Out Properties' shows 'Reconnect Timeout (ms)' as 60000 and a checkbox for 'Discard message once timeout occurs' which is unchecked.

- **No Retry**
Rhapsody does not attempt to reconnect to the external system.
- **Immediate**
Attempt to reconnect to the external system immediately. If the attempt fails, repeat the re-connection attempt up to the number of times specified by the Number of Retries property.
- **Linear**
Attempt to reconnect after a fixed delay specified by the Initial Retry Delay period. If multiple retries are specified, the delay between each attempt is specified by the Message Retry Delay period.
- **Exponential**
Attempt re-connection after an exponentially increasing delay interval. The delay between each connection attempt will be double the previous wait time until the maximum delay interval is reached.

Initial Retry Delay (ms)

This parameter establishes the delay (in milliseconds) before the first and subsequent re-connection attempts are made. The minimum value for this parameter is 10ms.

Number of Retries

This parameter defines the number of times to attempt the re-connection before considering the connection failed and stopping the communication point. If the **Retry Type** is set to **Immediate**, the retry attempts are made with a delay set by the **Message Retry Delay** period (default: 1 second) up to the **Number of Retries** value.

To prevent the communication point from stopping after too many failed connections, set the number of retries to **Infinite** (-1). The communication point never stops and continually attempts to connect to the service. This option can be used when long outages are anticipated and is the recommended option to make the communication more tolerant of client failures or poor configuration.

Maximum Retry Delay (ms)

This parameter is only applicable when the **Retry Type** is set to **Exponential**. It determines a cut-off period at which point the delay does not increase any further. The minimum value for this parameter is 1000ms. If set to **0**, no maximum retry delay applies.

The **Maximum Retry Delay** is normally set to a greater value than the **Initial Retry Delay**. If the maximum retry delay is less than the initial retry delay, the maximum retry delay will be used as the retry delay period.

Retries Per Message

This parameter sets the maximum number of retries allowed per message when the operation fails due to a connection either being closed or when a connection error occurs. If set to **Infinite** (-1), per message retries are disabled for this communication point.

Retries Exceeded Action

This parameter sets the action to be taken by the communication point when the **Retries per Message** have been reached for a given message.

Included Messages

This setting determines what types of messages will be considered when determining whether a connection error should be counted against the message or not. This option is not available for communication points in either Output or Bidirectional mode because only the outgoing messages can be included. For Out->In mode, the available options are:

- **All Messages**

A connection error can be counted against the maximum number of retries for a single message if the connection error occurs while sending the output message, waiting for the response, or while receiving the response.

- **Response Messages Only**

A connection error is counted against the maximum number of retries for a single message only if the connection error occurs while waiting for the response or while receiving the response.

Connection Retry Examples

Here are two examples that compare how the **Linear** and **Exponential** retry types function.

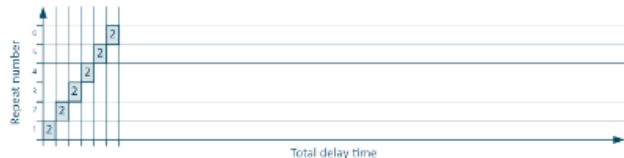
Linear retry type

This example uses the linear retry model.

The **Initial Retry Delay** is set to 2 seconds.

Connection Retries			
Retry Type:	Linear	Initial Retry Delay (ms):	2000
Number of Retries:	6	Maximum Retry Delay (ms):	300000
Retries per Message:	Infinite	Message Retry Delay (ms):	1000
Retries Exceeded Action:	Shut down communication point		
Redirect Target:	Select Communication Point		
Included Messages:	All Messages		

Notice how each connection retry attempt occurs after the same delay. Retry attempts will be made at this rate until the communication point is able to make a new connection or the **Number of Retries** value is exceeded.



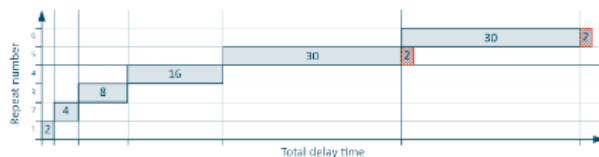
Exponential retry type

This example uses the exponential model.

The **Initial Retry Delay** is set to 2 seconds. The **Maximum Retry Delay** is set to 30 seconds.

Connection Retries			
Retry Type:	Exponential	Initial Retry Delay (ms):	2000
Number of Retries:	6	Maximum Retry Delay (ms):	30000
Retries per Message:	Infinite	Message Retry Delay (ms):	1000
Retries Exceeded Action:	Shut down communication point		
Redirect Target:	Select Communication Point		
Included Messages:	All Messages		

Notice how each subsequent retry attempt waits twice as long until the **Maximum Retry Delay** is reached. When this value is reached, each retry occurs after the same delay time until the **Number of Retries** is exceeded or the communication point can establish a new connection.



When using the Exponential retry method, the formula for the delay between retries is $\text{Retry Delay} = \text{Initial Retry Delay} \times 2^{(\text{current retry count})}$.

Assuming the Retry Type is Exponential, the Initial Retry Delay is 2000ms (2 seconds), the Number of Retries is 6 and the Maximum Retry Delay is 30,000ms (30 seconds), the delay period between retries will be:

1. $\text{retry} = 2 \times 2^0 = 2 \times 1 = 2$ seconds.
2. $\text{retry} = 2 \times 2^1 = 2 \times 2 = 4$ seconds.
3. $\text{retry} = 2 \times 2^2 = 2 \times 4 = 8$ seconds.

4. $\text{retry} = 2 \times 2^3 = 2 \times 8 = 16$ seconds.
5. $\text{retry} = 2 \times 2^4 = 2 \times 16 = 32$ seconds.
6. $\text{retry} = 2 \times 2^5 = 2 \times 32 = 64$ seconds.

But the Maximum Retry Delay has been set to 30 seconds so the 5th and 6th retry attempts will occur after 30 seconds, not 32 and 64 seconds.

Out-In and In-Out Properties

For the In->Out and Out->In synchronous modes, it is important that the communication point can reset if a message is not sent (for an In->Out mode communication point) or received (for an Out->In mode communication point). These properties let us control how a communication point resets to its initial state and what is done with any messages affected by the reset.

Properties

Depending on the mode of the communication point, one of the two panels illustrated below will be displayed on the **Connections** tab.

-
-

In->Out Properties



Included Messages: All Messages

Out->In Properties

Response Timeout (ms): 60000 ☐ Suspend message timeout once response is received

Retry Count: Infinite

Fail Action: Treat as error

Out->In Properties

☒ ☐

Response Timeout

The **Response Timeout** parameter specifies how long the communication point should wait for a response before resetting to the initial state.

① In In->Out mode this should be set to a value less than the client (or sending) application's timeout setting. For example, if the client application has a timeout of 60 seconds, then the response timeout could be set to 55 seconds. If the timeout in Rhapsody is higher than 60 seconds, then there is a possibility that the client application could time out and unnecessarily resend a message before receiving a response from Rhapsody. The resulting duplicated message could have a negative impact on downstream systems.

In Out->In mode this should be set to a value greater than the server (or receiving) application's timeout setting. For example, if the server application has a timeout of 60 seconds, then the response timeout could be set to 65 seconds. If the timeout in Rhapsody is less than 60 seconds, then there is a possibility that the communication point could time out and unnecessarily resend a message before receiving a response from the server application. The resulting duplicated message could have a negative impact on downstream systems.

Rhapsody supports the setting of a [Time-To-Live\(opens in a new tab\)](#) value on a message. When the Time-To-Live value on a message is exceeded, it will be discarded without being delivered to a destination. The following options can be used to control how Time-To-Live functions when using either the In->Out or Out->In modes:

- **Discard message once timeout occurs** (In->Out mode only)
When selected, received messages will have a time-to-live equal to the **Response Timeout** setting. This prevents any additional processing from being performed on a message which would be sent to the communication point after the **Response Timeout** has occurred.

- **Suspend message timeout once response is received** (Out->In mode only)
When selected, the time-to-live for a message received by the communication point is disabled. This ensures that processing of a response from a server completes, even if it took longer than expected for the response to be received.

Retry Count

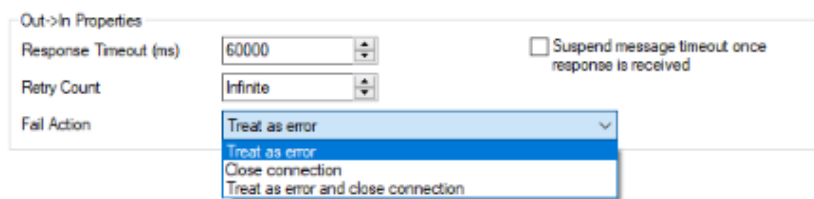
(Out->In mode only)

This parameter determines how many times the outbound message should be resent while waiting for a response. When set to **Infinite**, the communication point retries the same message indefinitely.

Fail Action

(Out->In mode only)

The action is executed when the maximum number of resend attempts has been reached and the **Response Timeout** has occurred.



Out->In Properties

Response Timeout (ms) 60000

Retry Count Infinite

Fail Action

☐ Suspend message timeout once response is received

Treat as error

Treat as error

Close connection

Treat as error and close connection

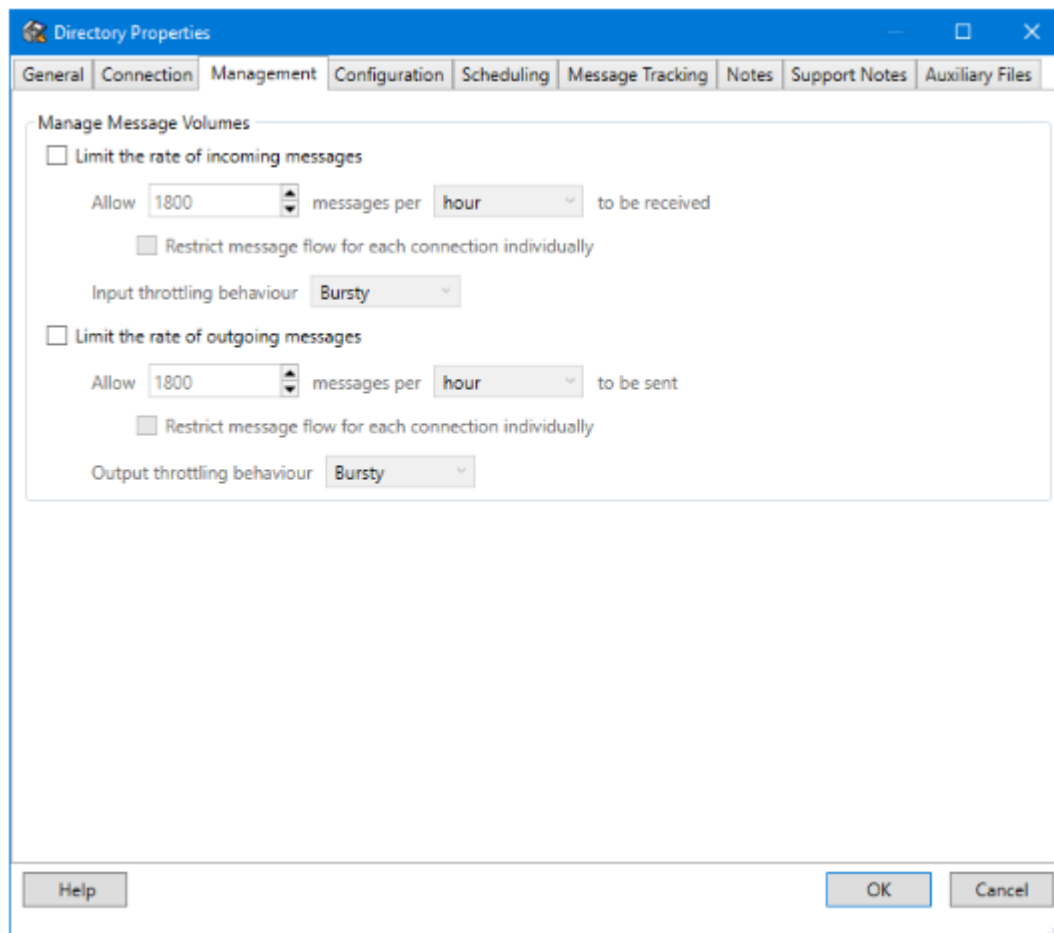
 The two **Treat as error** options make use of the communication point's error connector (part of the Continue routing on failure feature). These can be used to perform additional processing on messages that fail to generate a timely response from the destination system.

Management Tab

The **Management** tab provides a method of controlling the rate at which messages are sent or received in situations where processing queues back up for input messages or a downstream system cannot accept messages as rapidly as the engine can send them.

- For input communication points, the rate controls how often the communication point reads messages from the connection. It limits the rate at which the input queue grows.
- For output communication points, the rate controls the message release from the output communication point, meaning the output queue may back up.

The rate of input and output messages can be set independently and can be applied either to message traffic through the communication point or through each connection independently.



Throttling Behavior Modes

- **Restrict message flow for each connection individually**

If selected, input or output throttling will apply to each of the communication point's connections rather than to the communication point. If a communication point has two connections with each limited to 10 messages per minute, the communication point will be processing messages at a rate of 20 per minute.

When message throughput is throttled, two operating modes are possible:

- **Bursty**

The specified number of messages are sent at the start of the interval. For example, if the rate is sent at 10 messages per hour, the first 10 messages will be sent as soon as they are available and with a very small time gap between messages.

- **Linear**

The specified number of messages are sent at equal intervals throughout the time period.

Configuration Tab

The **Configuration** tab contains all the settings unique to each type of communication point. For example, the Directory communication point contains settings to specify the directory for messages.

Connection Mode:

Properties with an asterisk (*) are required.

Property	Value
Input Directory *	c:\testing\systems\SimplePAS ORU\
Input Control File Mode	☐
Input Control File Extension *	
Pattern Type	Wildcards
Pattern	*
Exclude Pattern Type	Wildcards
Exclude Pattern	
Input Order	Least recently modified first
Comparison Script *	
Refresh Rate	5000
Receive Rate	0
Move Input Files	☐
Processed Directory *	

Where to look for files/messages to process.

Help OK Cancel

Required properties are identified with an asterisk to the left of their name. A communication point cannot be started and begin processing messages if a required property value is blank. Greyed out properties are not applicable and can be ignored, even if they are also marked as required. An example of this is the **Input Control File Extension** property, which is only required if the preceding **Input Control File Mode** property has been set to **True**.

Configuration fields do not support concatenation of property values and strings. If concatenation is required, this needs to be done in a filter and stored as a message property prior to the message reaching the output communication point. This new message property is referred to in the communication point configuration. For example, if you wanted to append a message type to a patient identifier for the **BaseFilename** property, this would require creating a property on the route that concatenates the two items.

Selecting a property displays a summary of the property's usage at the bottom of the screen. This will frequently provide you with enough information to correctly configure the property.

When the engine and IDE are running on different machines, communication point settings (such as a directory path or TCP port number) refer to locations on the machine running the engine.

Additional Tabs

Scheduling Tab

The **Scheduling** tab allows its activity to be restricted to specific times, for example, to process messages overnight rather than during the day, or to stop communication points to allow a regularly occurring external process (such as a system backup) to take place.

The schedule can be defined as a daily or weekly schedule.

Directory Properties

General | Connection | Management | Configuration | Scheduling | Message Tracking | Notes | Support Notes | Auxiliary Files

☐ Enable scheduling for this communication point

☐ Daily

Start: 12:00 am

Stop: 12:00 am

☐ Monday ☐ Tuesday ☐ Wednesday
☐ Thursday ☐ Friday ☐ Saturday
☐ Sunday

☐ Weekly

Start: 12:00 am on Sunday

Stop: 12:00 am on Sunday

☐ Advanced

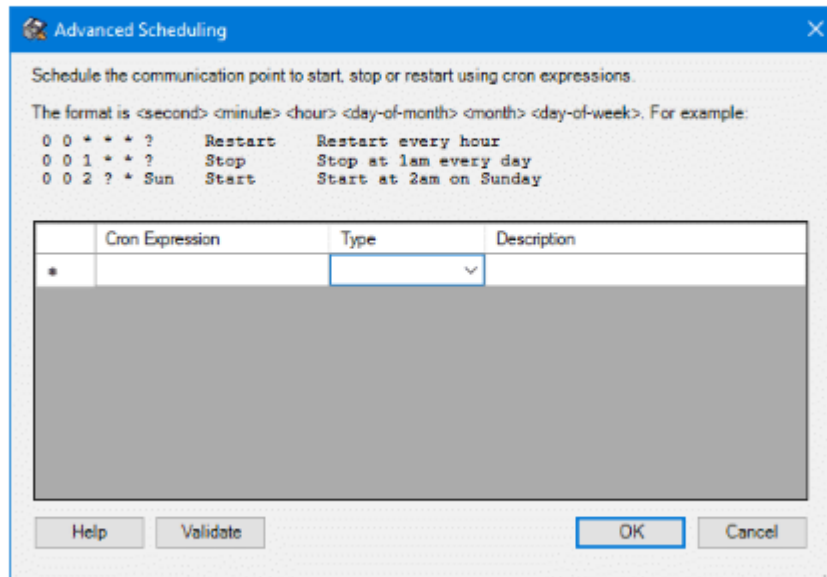
Define Advanced Scheduling

Not Configured.

Help OK Cancel

① If scheduling information has been enabled for a communication point then subsequently disabled, it will be remembered and recalled if scheduling is re-enabled at a later date.

Where more control is required over scheduling, [Cron expressions](#) can be entered via the **Define Advanced Scheduling** link.



Advanced Scheduling Dialog

Message Tracking Tab

A communication point's Message Tracking functionality allows the engine to match messages sent by the engine with their responses. Message Tracking uses specific field values present in both messages (normally a message identifier) to perform the matching.

The schemes also support management of retries if the response message is not received in a timely manner. They also control the message behavior if an error occurs, or no response is received and the retries are exhausted, such as discarding the message or re-sending the message.

The following example shows a Message Tracking scheme set up to track HL7 V2.4 ADT messages.

Directory Properties

General | Connection | Management | Configuration | Scheduling | **Message Tracking** | Notes | Support Notes | Auxiliary Files

You can specify a scheme according to which messages will be correlated with responses, configured actions executed based on response type and cases handled, such as timeouts and invalid messages.

☒ Enable message tracking for this communication point

Scheme

Scheme: Academy PAS

View Override New

Summary

Definition: Academy-HL7v2.4-ADT.s3d

Tracked messages: ADTA01, ADTA03, ADTA28, ADTA29, ADTA30, ADTA31

Response messages: ACK, ACKA01, ACKA03, ACKA28, ACKA29, ACKA30, ACKA31

Response codes: AA, AE, AR

Response timeout: 60 s

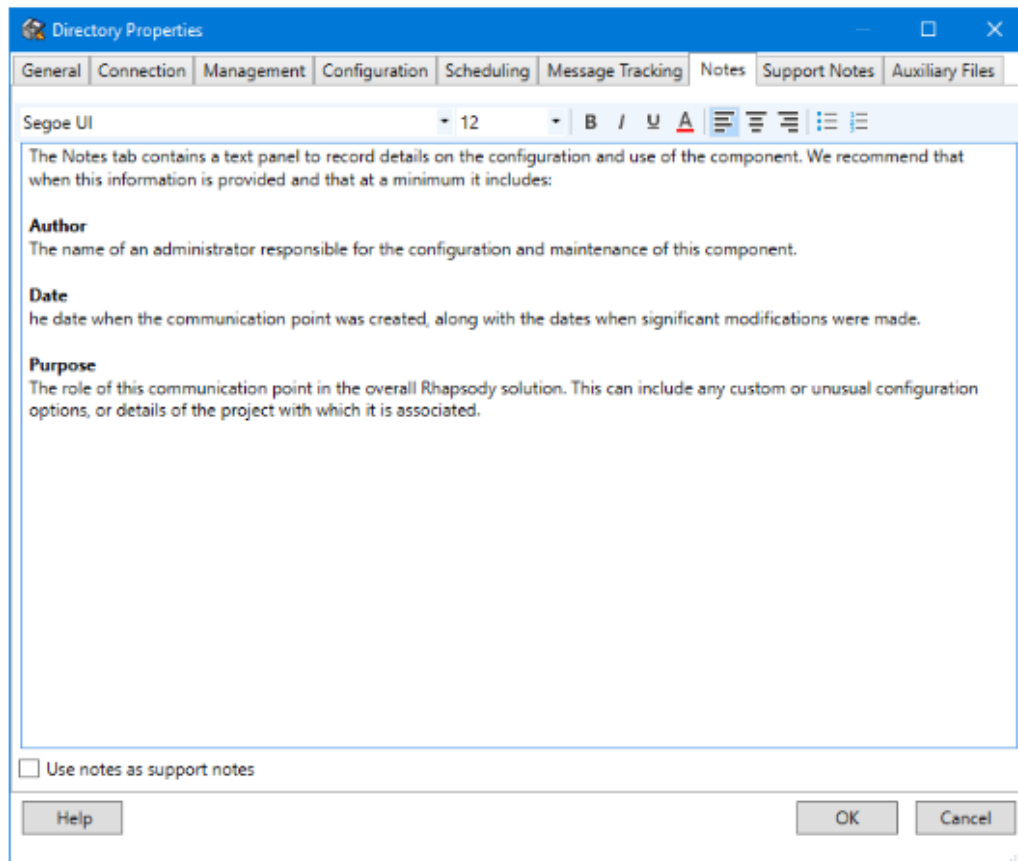
Help OK Cancel

 Additional information and illustrative exercises for this topic are included in the *Rhapsody Professional* course.

Notes Tab

The **Notes** tab contains a text panel to record details on the configuration and use of the component.

Formatting can be added to notes to make them easier to read. Use a common format for all notes to make them easier to read at a glance. The contents of the **Notes** tab are available from the IDE and in any generated PDF documentation.

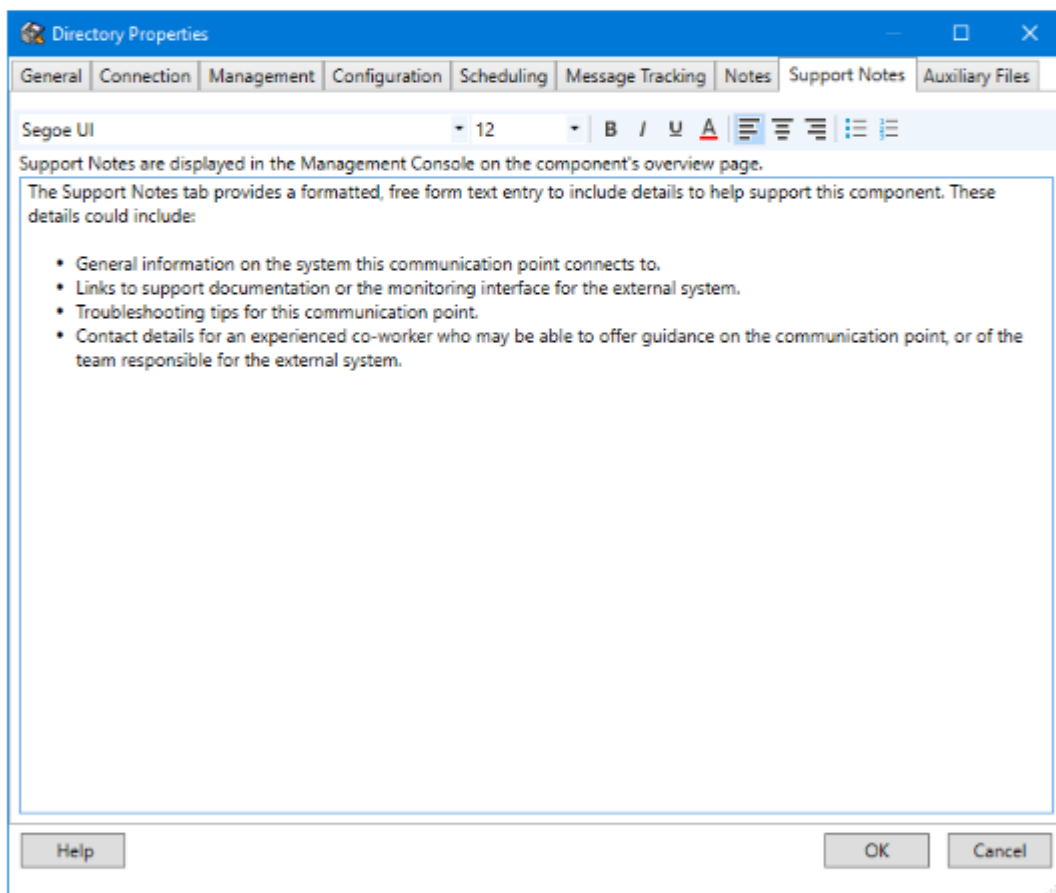


Where your notes are suitable for use as support notes, the **Use notes as support notes** checkbox can be selected. This disables the contents of the **Support Notes** tab and uses the contents of the **Notes** tab on the **Support Notes** panel on the component's **Overview** window in the Management Console.

Support Notes Tab

The **Support Notes** tab provides a formatted, free form text entry to include details to help support this component. These details could include:

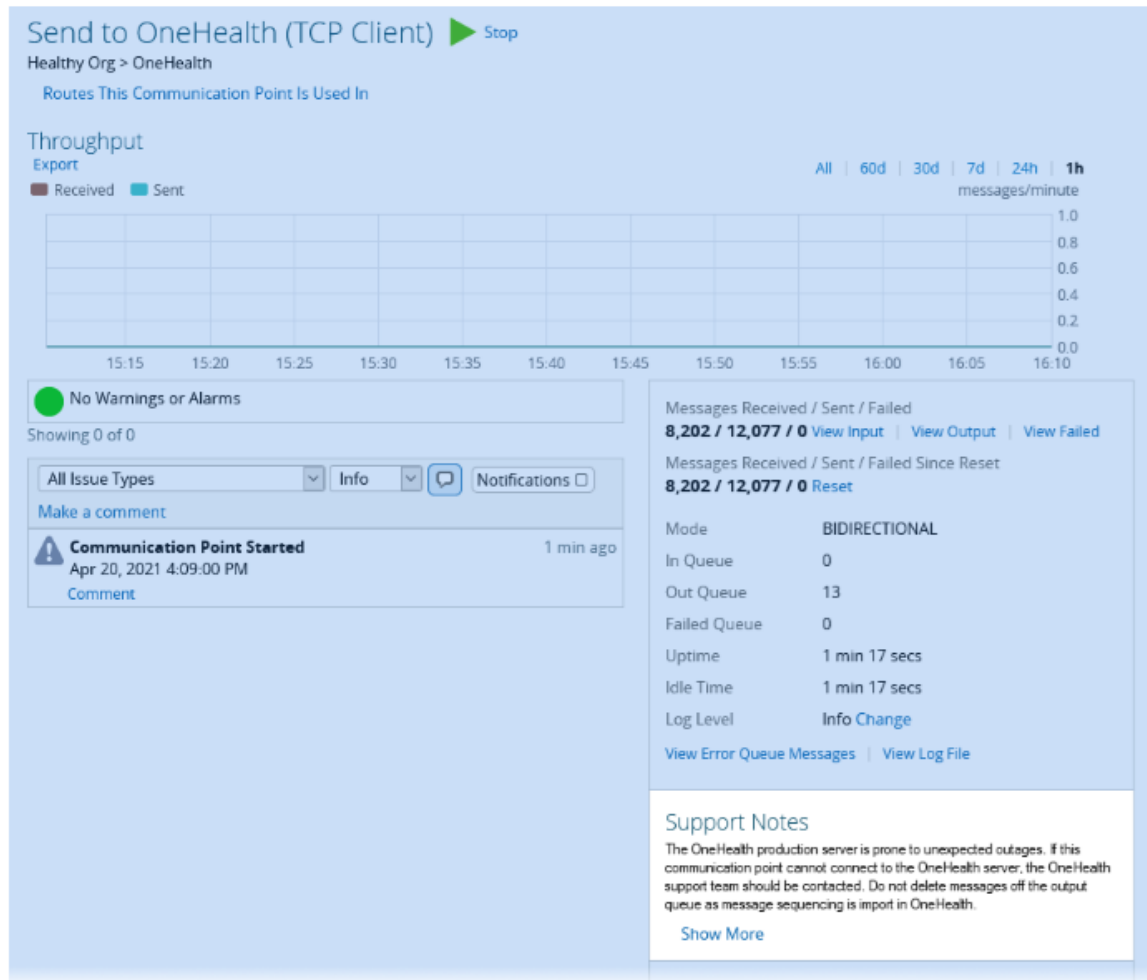
- General information on the system this communication point connects to.
- Links to support documentation or the monitoring interface for the external system.
- Troubleshooting tips for this communication point.
- Contact details for an experienced co-worker who may be able to offer guidance on the communication point, or of the team responsible for the external system.



 We recommend you create support notes for all communication points that connect to an external system. This helps provide relevant information to anyone who needs to diagnose a connection problem.

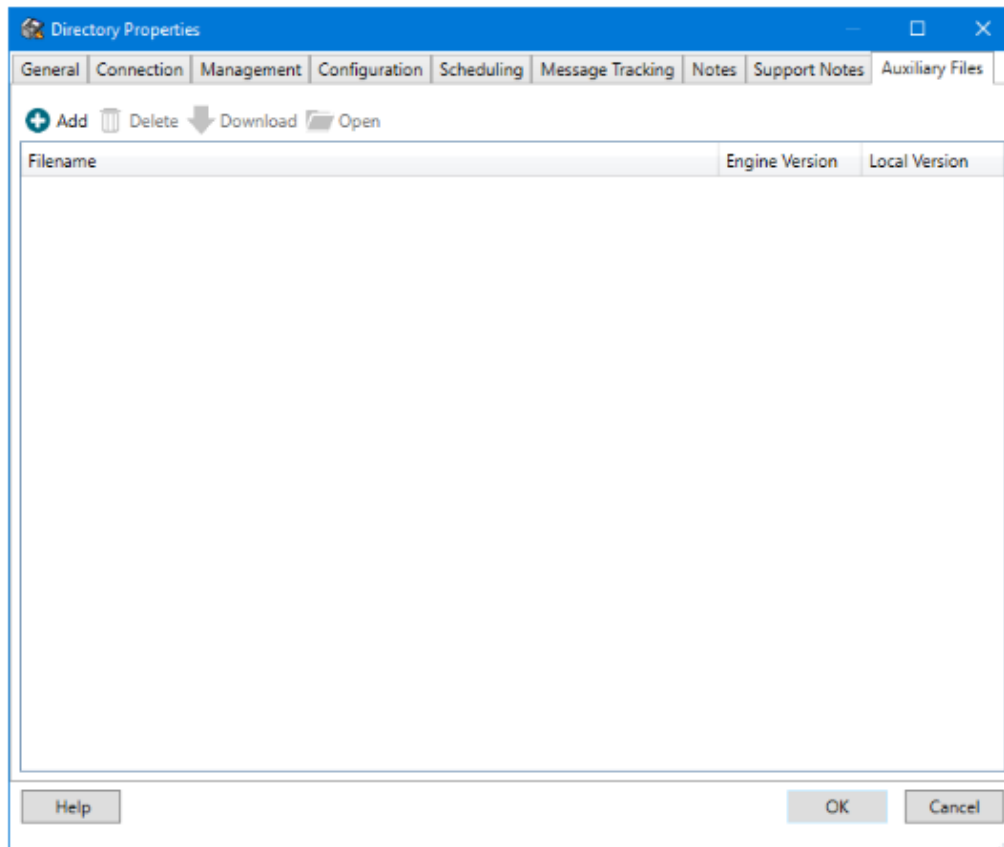
View support notes

The information entered on a communication point's **Support Notes** tab is displayed on the Overview window in the Management Console.



Auxiliary Files Tab

The **Auxiliary Files** tab manages files associated with the communication point configuration. Examples of these files include the body template file associated with a Timer communication point and database configuration files associated with a Database communication point.



The communication point must be checked in after making a change to the auxiliary files for the changes to appear. This is indicated on the tab when both the Engine and Local versions of the file are identical.

Exercise - Common Properties

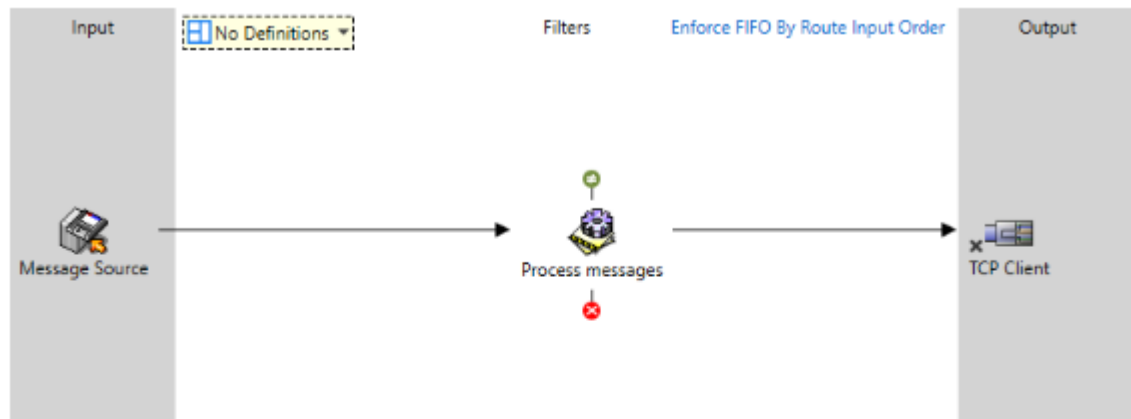
This exercise uses the Directory communication point and introduces the TCP Client communication point. Its purpose is to illustrate the common communication point configuration tasks with a focus on two key properties: **Retry Behavior** and **Throughput Management**.

Create the Environment

Begin by creating the following new route (this is similar to routes you have created in previous exercises).

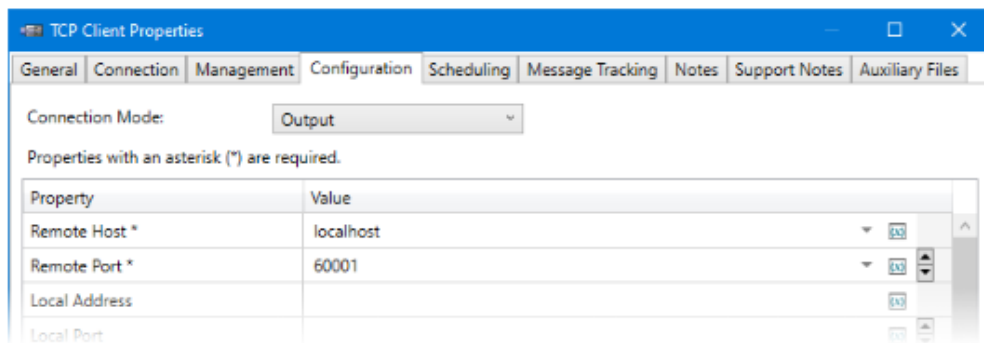
The Directory communication point should be configured in Input mode and should point to a convenient local directory into which the test files can be placed. All other options can be left with their default values.

The TCP Client communication point should be configured in Output mode so that it can be added to the output side of the route.



From the **Properties** dialog for the TCP Client communication point, select the **Configuration** tab. Enter the following settings:

- **Remote Host:** `localhost`
This forces the communication point to attempt to connect to a server (which does not exist) running on the same machine as the Rhapsody engine.
- **Remote Port:** `60001`
60001 is a network port which is not normally used by any applications, and which therefore should not generate a conflict in this exercise.



Select **OK** to complete the configuration. Check in the route and two communication points but do not start them yet.

Exercise Tasks

This exercise uses the TCP Client communication point you just created to examine the retry behavior of a communication point when a connection cannot be established with a server at a network level. A communication point's **Connection** tab supports three models to control retry (or re-connection) behavior if a connection fails to be maintained at the network level. In this exercise, we will test the following three approaches:

- **Immediate**
If the connection fails, immediately attempt to re-connect. Repeat for the number of attempts defined by the **Number of Retries** parameter.
- **Linear**
If the connection fails, wait for the period defined by the **Initial Retry Delay** and attempt to re-connect. Repeat for the number of attempts defined by the **Number of Retries** parameter.
- **Exponential**
If the connection fails, attempt reconnection after an exponentially increasing delay until either the connection is re-established or the number of retry attempts is reached, as defined by the **Number of Retries** parameter.

Immediate Retry Exercise Tasks

1. 1

Configure the following in the **Connection** tab of the TCP Client communication point.

- **Retry Type:** Immediate

- **Number of Retries:** 5
- **Retries Per Message:** Infinite

2. 2

Check in the changes and start the communication point. It will eventually fail (after 5 retries) and shut down. This is indicated by the red exclamation mark displayed beside the communication point in the **Workspace**.

3. 3

Open the Management Console and, from the **System State** menu, select and examine the **System Log**.

The **System Log** page will display the attempt to start the communication point, the initial failure followed by the retries, and the subsequent stopping of the component once the connection retry limit has been exceeded.

Education
7.4.0 (Development)

Administrator 10-Sep-2024 14:09

Rhapsody
Integration Engine

Monitoring
Engine Monitor
Communication Points
Routes
Web Services
REST Clients
System Monitor
Error Queue
Hold Queue
Ports in Use

Find Messages
Notifications
Management
System State
System Log
Audit Log
Threads
Export Tools
User Sessions
Database Key Values
JavaScript Counters

Reports

System Log Export Logs

From
To
Log Level
Number of results
Component

Search

Date	Level	Component	Details
Oct 10, 2022 15:56:44.357	INFO	System	Communication point 'TCP Client' has stopped in error: Connection retries exceeded Details: IO error reconnecting.
Oct 10, 2022 15:56:44.356	WARN	TCP Client	Communication point stopped in error: Connection retries exceeded
Oct 10, 2022 15:56:44.353	ERROR	TCP Client	Exceeded connection retry limit, shutting down communication point Show error details
Oct 10, 2022 15:56:43.903	INFO	TCP Client	Communication Point Connection Failed (4 more) Show error details
Oct 10, 2022 15:56:43.903	INFO	TCP Client	Communication Point Connection Failed Show error details
Oct 10, 2022 15:56:43.902	INFO	System	Communication point 'Message Source' has been started: Communication point started by user 'Administrator'.
Oct 10, 2022 15:56:43.902	INFO	System	Communication point 'TCP Client' has been started: Communication point started by user 'Administrator'.
Oct 10, 2022 15:56:43.901	INFO	Message Source	Communication point started
Oct 10, 2022 15:56:43.901	INFO	TCP Client	Communication point started
Oct 10, 2022 15:56:43.900	INFO	System	Route 'Exercise' has been started: Route started by user 'Administrator'.
Oct 10, 2022 15:56:43.896	INFO	Exercise	Route started
Oct 10, 2022 15:56:36.393	INFO	System	The configuration has been changed by user 'Administrator'. Modified Communication Points: TCP Client (Exercises/REN-105/Common Properties)

The **Event** timestamp of the events is shown in seconds (with milliseconds following the decimal point).

- Review the timestamps for the sequence and note the time interval between starting the communication point, the time delay recorded for the initial connection failure and the time delay recorded for shutting down the communication point.

What is the total time in seconds between startup and shutdown of the communication point for the immediate retry model?

- ☐ 0 seconds
- ☐ Between 0.001 and 15 seconds
- ☐ Between 15.001 and 30 seconds
- ☐ Between 30.001 and 45 seconds
- ☐ Between 45.001 and 60 seconds
- ☐ Greater than 60 seconds

SUBMIT

5

Open the **Monitoring** menu on the left of the screen and select **Communication Points**. Select the TCP Client communication point to view its **Details** screen.

6

Examine the log window. The image below is similar to what you would expect to see:

6

Examine the log window. The image below is similar to what you would expect to see:

Communication Point Stopped In Error 2 mins ago
 Jun 24, 2019 11:59:45 AM
[Comment](#) [Dismiss Alert](#) [Suspend Notifications](#) [Assign](#)

Showing 1 of 1

All Issue Types Info Notifications

Make a comment

- ☐ **Communication point stopped in error: Connection retries exceeded** 2 mins ago
- ☐ **Exceeded connection retry limit, shutting down communication point** 2 mins ago
[Show error details](#)
- ☐ **Communication Point Connection Failed** 2 mins ago
[Show error details](#)
- ☐ **Communication Point Connection Failed** 2 mins ago
[Show error details](#)
- ☐ **Communication Point Connection Failed** 2 mins ago
[Show error details](#)

7

On the **Exceeded connection retry limit, shutting down communication point** option, click the **Show error details** link and observe the error detail.

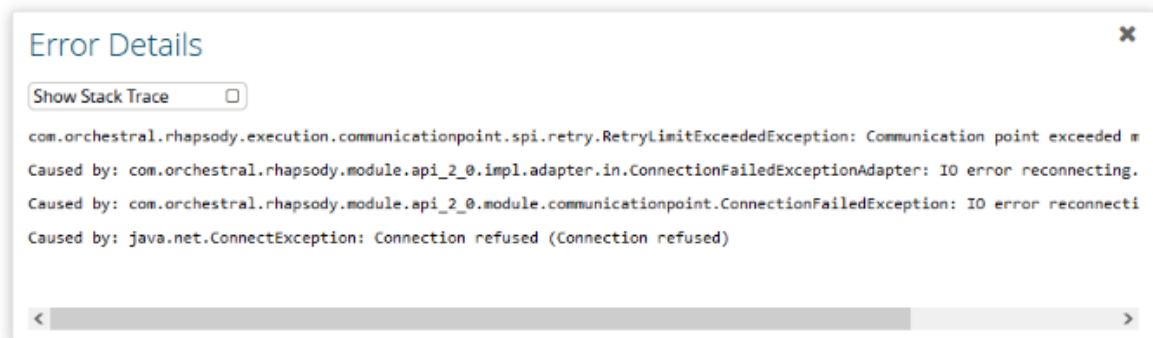
Error Details

Show Stack Trace

```
com.orchestral.rhapsody.execution.communicationpoint.spi.retry.RetryLimitExceededException: [com.orchestral.rhapsody.execution.communicationpoint.spi.retry.LinearConnectionRetryStrategy.connectionFailed
    at com.orchestral.rhapsody.execution.communicationpoint.spi.retry.LinearConnectionRetryStrategy.connectionFailed
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor.run(CommunicationPointExecutor.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor$ControllerThread.execute(CommunicationPointExecutor.java:110)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.AbstractCommunicationPointThread.run(AbstractCommunicationPointThread.java:40)
Caused by: com.orchestral.rhapsody.module.api_2_0.impl.adapter.in.ConnectionFailedExceptionAdapter: [com.orchestral.rhapsody.module.api_2_0.impl.adapter.in.ConnectionFailedExceptionAdapter.getConnection(CommunicationPointAdapter.java:100)
    at com.orchestral.rhapsody.module.api_2_0.impl.adapter.in.ConnectionFailedExceptionAdapter.getConnection(CommunicationPointAdapter.java:100)
    at com.orchestral.rhapsody.component.type.spi.CommunicationPointProxy.getConnection(CommunicationPointProxy.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointInternalImpl.getConnection(CommunicationPointInternalImpl.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointInternalImpl.getConnectionInternal(CommunicationPointInternalImpl.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor.getConnectionInternal(CommunicationPointExecutor.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor.run(CommunicationPointExecutor.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor$ControllerThread.execute(CommunicationPointExecutor.java:110)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.AbstractCommunicationPointThread.run(AbstractCommunicationPointThread.java:40)
Caused by: com.orchestral.rhapsody.module.api_2_0.module.communicationpoint.ConnectionFailedException: [com.orchestral.rhapsody.module.api_2_0.module.communicationpoint.ConnectionFailedException.getConnection(TCPClientTransportConnection.java:100)
    at com.orchestral.rhapsody.module.api_2_0.module.communicationpoint.ConnectionFailedException.getConnection(TCPClientTransportConnection.java:100)
    at com.orchestral.rhapsody.resource.tcp.TCPClientTransport$TCPClientTransportConnection.reconnect(TCPClientTransportConnection.java:100)
    at com.orchestral.rhapsody.resource.tcp.TCPClientTransport$TCPClientTransportConnection.<init>(TCPClientTransportConnection.java:100)
    at com.orchestral.rhapsody.resource.tcp.TCPClientTransport.getConnection(TCPClientTransport.java:24)
    at com.orchestral.rhapsody.module.api_2_5.module.communicationpoint.ProtocolTransportCommunicationPoint.getConnection(ProtocolTransportCommunicationPoint.java:100)
    at com.orchestral.rhapsody.module.api_2_5.module.communicationpoint.ProtocolTransportCommunicationPoint.getConnection(ProtocolTransportCommunicationPoint.java:100)
    at com.orchestral.rhapsody.module.api_2_0.impl.adapter.in.CommunicationPointAdapter.getConnection(CommunicationPointAdapter.java:100)
    at com.orchestral.rhapsody.component.type.spi.CommunicationPointProxy.getConnection(CommunicationPointProxy.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointInternalImpl.getConnection(CommunicationPointInternalImpl.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointInternalImpl.getConnectionInternal(CommunicationPointInternalImpl.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor.getConnectionInternal(CommunicationPointExecutor.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor.run(CommunicationPointExecutor.java:100)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.CommunicationPointExecutor$ControllerThread.execute(CommunicationPointExecutor.java:110)
    at com.orchestral.rhapsody.execution.communicationpoint.spi.AbstractCommunicationPointThread.run(AbstractCommunicationPointThread.java:40)
Caused by: java.net.ConnectException: Connection refused (Connection refused)
    at java.net.PlainSocketImpl.socketConnect(Native Method)
    at java.net.PlainSocketImpl.doConnect(PlainSocketImpl.java:350)
    at java.net.PlainSocketImpl.connectToAddress(PlainSocketImpl.java:206)
    at java.net.PlainSocketImpl.connect(PlainSocketImpl.java:188)
    at java.net.SocksSocketImpl.connect(SocksSocketImpl.java:392)
    at java.net.Socket.connect(Socket.java:607)
    at com.orchestral.rhapsody.resource.tcp.TCPClientTransport$TCPClientTransportConnection.connectToSocket(TCPClientTransportConnection.java:100)
```

 Depending on the resolution of your monitor and the complexity of the error, you might need to scroll to the right to see the full error details.

The **Show Stack Trace** button at the top of the Error Details screen can be used to hide the stack trace for the error. For many errors this makes the cause significantly easier to review.



Generally, the last **Caused by:** entry on the Error Details will reveal the root cause of the fault.

Linear Retry Exercise Tasks

1

Re-configure the **Connection Retries** parameters as follows:

- **Retry Type:** Linear
- **Initial Retry Delay:** 5000 (ms)
- **Number of Retries:** 5
- **Retries Per Message:** Infinite

2

Check in the changes and start the communication point. It will eventually fail after 5 retries and shut down.

3

Open the Management Console and examine the **System Log**.

4

Review the timestamps for the sequence and note the time interval between starting the communication point, the time delay recorded for the initial connection failure, and the time delay recorded for shutting down the communication point.

What is the total time in seconds between startup and shutdown of the communication point for the linear retry model?

- ☐ Between 0 and 19.999 seconds
- ☐ Between 20 and 39.999 seconds
- ☐ Between 40 and 59.999 seconds
- ☐ Between 60 and 79.999 seconds
- ☐ Between 80 and 99.999 seconds
- ☐ 100 seconds or greater

Exponential Retry Exercise Tasks

1

Re-configure the **Connection Retries** parameters as follows:

- **Retry Type:** Exponential
- **Initial Retry Delay:** 5000 (ms)
- **Number of Retries:** 5
- **Maximum Retry Delay:** 30000 (ms)
- **Retries Per Message:** Infinite

2

Check in the changes and start the communication point. It will eventually fail after 5 retries and shut down.

3

Open the Management Console and examine the **System Log**.

4

Review the timestamps for the sequence and note the time interval between starting the communication point, the time delay recorded for the initial connection failure, and the time delay recorded for shutting down the communication point.

What is the total time in seconds between startup and shutdown of the communication point for the exponential retry model?

Enter the value you see, rounded to the closest 20 seconds.

Examples:

If you see a time of 15.762 seconds, you should enter *20* as your response.

If you see a time of 49.067 seconds, you should enter *40* as your response.

Type your answer here

Exercise - Input Throttling Behavior

Input Throttling controls the rate at which messages are released into a route by an input communication point. This functionality has two operating behaviors:

- **Bursty**
Releases the specified number of messages into the route at the start of the time interval. This results in a spiked pattern of message transmission.
- **Linear**
Spreads the message transmission equally across the transmission time interval, resulting in a relatively smooth pattern of message transmission

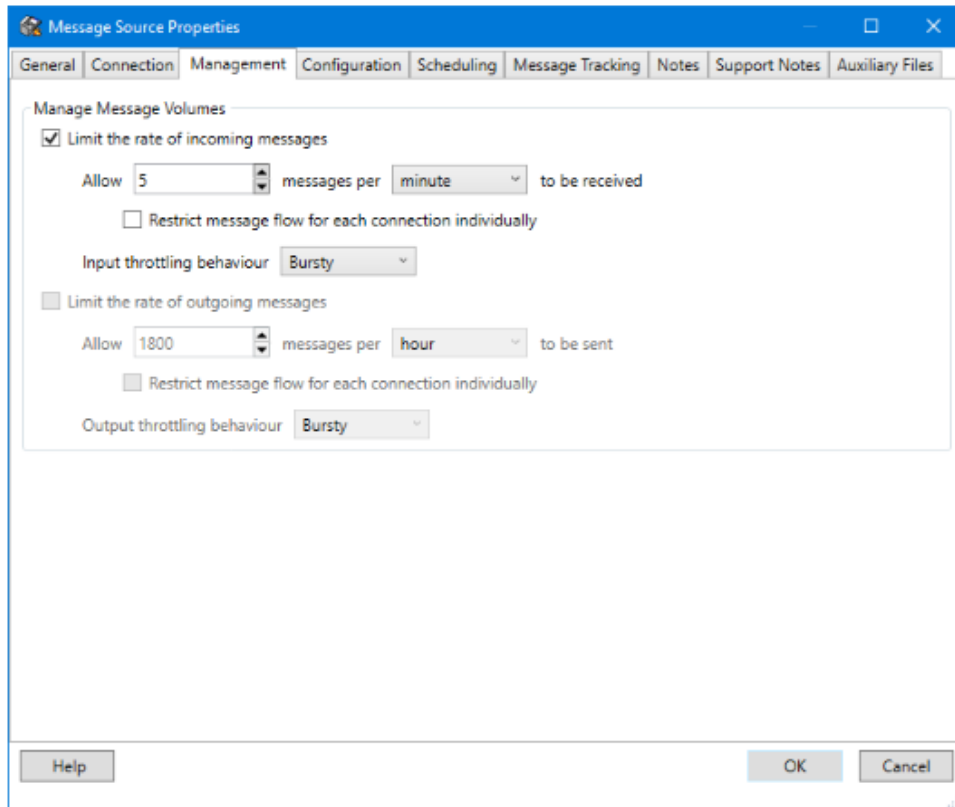
Resources

This exercise makes use of messages available in the in the .\Associate Files and Definitions\105 - Communication Points\Input Throttling Behavior folder in the resources ZIP file available at the start of the course.

Tasks

Tasks

- 1 Open the directory communication point's **Properties** dialog and switch to the **Management** tab. Configure it as shown below:

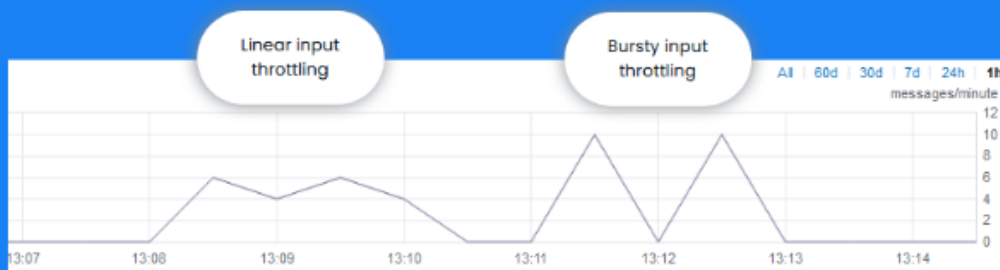


- 2 Ensure the **Input Throttling Behavior** is set to **Bursty**. Check in the change if one was made.
- 3 Copy the set of ten test messages into the communication point's input directory. Watch what happens to the messages.
- 4 Wait a couple of minutes to ensure all messages have been processed, then reopen the **Management** tab and change the throttling behavior to **Linear**. Check in the change.
- 5 Re-copy the ten test messages into the communication point's input directory. Watch what happens to the messages.

In Bursty mode, five of the test messages will be picked up and passed into the route. After a further minute, the remaining five messages will be picked up.

- 6 Open the Rhapsody Management Console. From the **Monitoring** menu, select **Routes**.
- 7 Locate and select the route created for this exercise to view its **Details** screen.
- 8 Review the **Throughput** graph for the Bursty and Linear throttling methods. Do the patterns match what you expected to see?

Model Answer



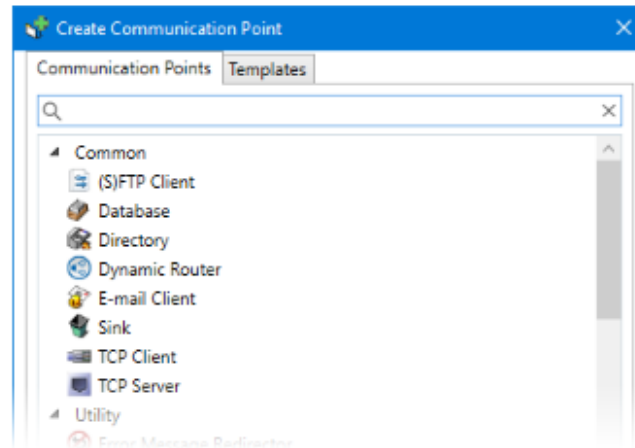
Linear mode

In Linear mode, the messages are picked up at 12 second intervals until they have all been sent for processing. This should result in a flatter graph showing the message's regular processing. This result may be more obvious if you increase the number of test messages and run the exercise for a greater period of time.

Bursty mode

In Bursty mode, five of the ten messages are immediately picked up by the communication point and passed into the route. One minute later, the remaining five are picked up. This should result in a graph with sharp peaks separated by one minute.

The most commonly used communication points in a Rhapsody solution are grouped together at the top of the list of communication points in the **Create Communication Point** dialog.



These communication points, along with the Timer communication point, and their common configuration parameters are described in this section.

i The Sink communication point has been described previously in this course and will not be covered again in this section.

Client vs Server

Many of the communication points include either a client or server suffix in their name. Some, such as the TCP communication points, have both client and server variants. These terms refer to how communication points establish connections to external systems.

- Server communication points listen for incoming connections from external systems once started.
- Client communication points attempt to establish a connection to a particular server when started or sending a message.

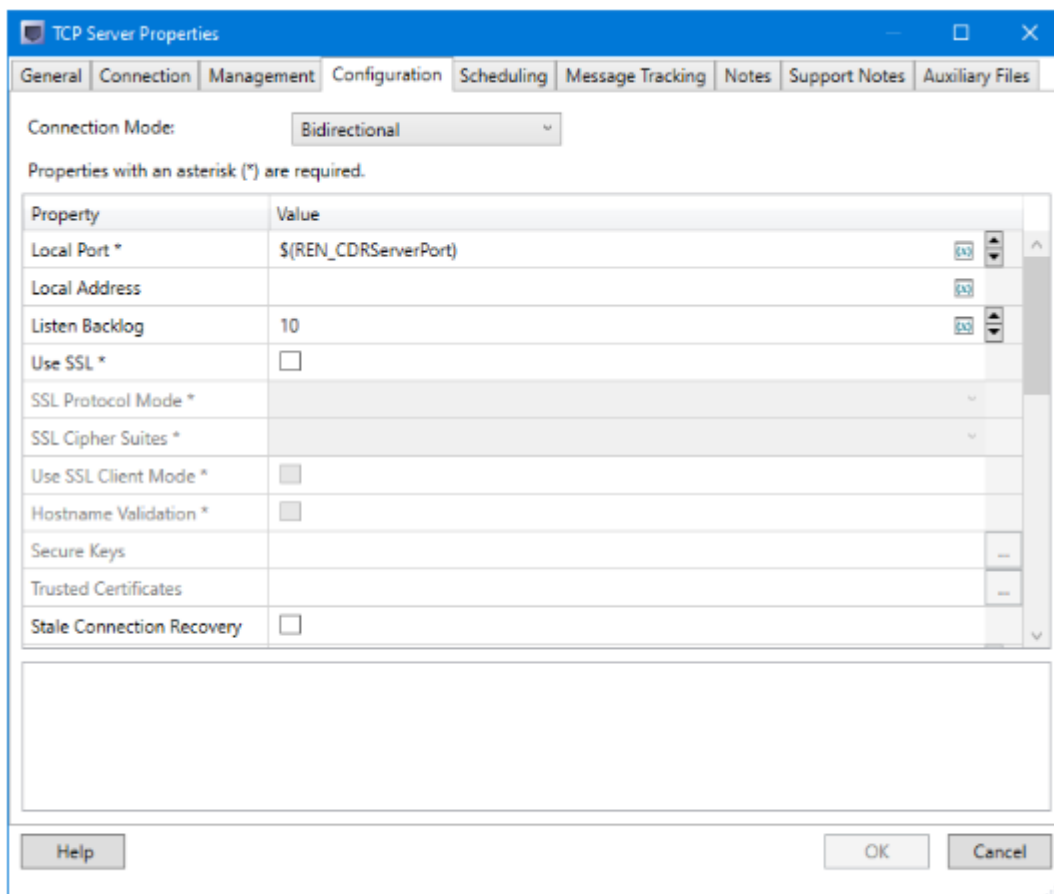
This behavior depends on the type of communication point and its configuration settings. In simple terms, a server listens for incoming connections, while a client connects to a server.

Examples

- Your browser can be thought of as a client that connects to the Academy server and receives responses that include the page you are currently viewing.
- A blood analyzer machine operates as a client connecting to a Rhapsody server communication point to send a sequence of laboratory messages.
- *Lesson 14 of 23*

• TCP Server

The TCP Server communication point opens the nominated **Local port** and listens for connections from remote client devices. The clients need to establish a TCP connection to the communication point before message transfer can occur.



The screenshot shows the 'TCP Server Properties' dialog box with the 'Configuration' tab selected. The 'Connection Mode' is set to 'Bidirectional'. A note states: 'Properties with an asterisk (*) are required.' Below this is a table of properties:

Property	Value
Local Port *	\$(REN_CDServerPort)
Local Address	
Listen Backlog	10
Use SSL *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Use SSL Client Mode *	☐
Hostname Validation *	☐
Secure Keys	
Trusted Certificates	
Stale Connection Recovery	☐

At the bottom of the dialog are buttons for 'Help', 'OK', and 'Cancel'.

At a minimum, you need to specify the **Local Port** property to define the TCP port number that the communication point listens on. The value of the port number can be selected arbitrarily in the range 1025 - 65335 (port numbers below 1025 are typically

reserved for system use). In general, you should avoid ports associated with well-known protocols. This property is normally specified using a Rhapsody variable.

By default, this communication point handles a connection from a single connecting device. This can be modified by changing the **Number of Connections** setting on the **Connection** tab. Connection tracking should be used to ensure a response goes back to the originating client.

Communication point options

The TCP Server communication point uses several options to control how it interacts with external systems.

Message wrapping

HL7 messages transmitted over TCP typically have special characters added before and after the content to allow the remote system to identify the start and end of the message. These characters are referred to as message wrapping. By default, the TCP communication points will use the HL7 Minimal LLP wrapping scheme to wrap all messages sent and received.

① Messages using the HL7 Minimal wrap scheme are preceded by the x0B character and followed with the x1C and x0D characters.

The full set of message wrapping models is:

- **Minimal** (default)
HL7 minimal LLP protocol.
- **User**
User-defined header and/or trailer.
- **Hybrid**
HL7 hybrid LLP protocol.
- **None**
No message wrapping performed.

The **Strip Wrapping** property, when enabled, removes the wrapping from incoming messages. This is enabled by default and is normally left unaltered.

Log connections

The TCP Server communication point provides support for debugging connection issues via the Connection Log. If **Log Connections** is enabled, each connection attempt will be logged. The data can be logged either as text or hexadecimal, with the latter useful for investigating issues with the message wrappings.

Supported Modes

The TCP Server communication point supports all modes, however it will generally be used in either In->Out or Bidirectional modes only.

Lesson 15 of 23

TCP Client

The TCP Client communication point is designed to connect to a remote (server) system and then send or receive messages from it.

The screenshot shows the 'TCP Client Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Out->In'. A note states: 'Properties with an asterisk (*) are required.' The following table lists the properties and their values:

Property	Value
Remote Host *	\$(TCP_Remote_Host)
Remote Port *	\$(TCP_Remote_Port)
Local Address	
Local Port	
Use SSL *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Use SSL Client Mode *	☐
Secure Keys	...
Trusted Certificates	...
Connection Timeout (ms) *	300000
Dynamic Connection Failed Action	Treat as message error
Log Connections *	☐

At the bottom of the dialog are buttons for 'OK', 'Cancel', 'Apply', and 'Help'.

At a minimum, the **Remote Host** and **Remote Port** properties need to be configured. These define the location of the server that the communication point will connect to. The **Remote Host** property identifies the host to which the connection will be made and needs to be configured with a valid hostname or IP (network) address. The **Remote Port** identifies the network port which will be used for the connection. It is Best Practice that both the **Remote Host** and **Remote Port** properties be configured using Rhapsody variables.

Normally, Rhapsody selects the appropriate network interface to use for the connection, but in rare cases on a system with multiple interfaces, the interface to use can be specified with the **Local Address** parameter. Similarly, the **TCP port** used to originate the connection is usually randomly assigned in the range 1025 to 65535, but this may be locked to a specific port using the **Local Port** parameter.

Communication point options

The TCP Client communication point uses the same options as the previously discussed TCP Server for message wrapping and connection logging.

Connection mode

The **Connection Mode** parameter allows the connection to be permanently maintained in cases where messages are transmitted frequently, which saves the overhead of creating a new TCP connection for each message.

When the message traffic is infrequent, the **Connection Mode** can be set to disconnect when not sending a message.

Supported Modes

The TCP Client communication point supports all modes, however it is unlikely that you will need to use a mode other than Out->In or Bidirectional. The Out->In mode is frequently used when transmitting HL7 messages and a message must be acknowledged before the next message can be transmitted.

Lesson 16 of 23

Directory

The Directory communication point sends and receives files from a directory on the Rhapsody server or an attached network file share. It supports all connection modes.

File Permissions

When reading a file from disk, Rhapsody requires both read and write access to the file. If this level of file access is not provided, the file will not be processed by Rhapsody. When a file is written to disk, write access is needed on the directory where the file will be saved. If Rhapsody is unable to write a file to disk, it will remain in the output queue on the communication point.

When a read or a write operation fails due to a permissions issue, a log message indicating the reason for failure is generated for the component. These can be viewed from the Management Console..

Input Mode

In Input mode, the Directory communication point polls the configured directory at regular time intervals (5 seconds by default). When a new file is detected, its content is copied into a message for processing by the route. The input file is then either deleted or moved to a nominated processed files directory. The engine therefore requires both read and write access to the input directory to process messages correctly. Subfolders and files in any subfolders are ignored.

The screenshot shows the 'Incoming Patient Notifications Properties' dialog box with the 'Configuration' tab selected. The 'Connection Mode' is set to 'Input'. Below this, a note states 'Properties with an asterisk (*) are required.' A table lists various properties and their values:

Property	Value
Input Directory *	\$(SimplePAS_NotificationDir)\Patient
Input Control File Mode	☐
Input Control File Extension *	
Pattern Type	Wildcards
Pattern	*
Exclude Pattern Type	Wildcards
Exclude Pattern	
Input Order	None
Comparison Script *	
Refresh Rate	5000
Receive Rate	0

Below the table is a text area labeled 'Where to look for files/messages to process.' At the bottom of the dialog are buttons for 'Help', 'OK', and 'Cancel'.

Processing Selected Files

There are times when the directory communication point should only process files with a particular filename. There are four **Pattern** options that allow selected files to be included or excluded from processing by the communication point.

The **include** and **exclude** pattern type can be specified as either a Wildcards or Regular expression.

For example, to restrict processing to files with a suffix of .txt, a Wildcards type pattern of *.txt would be used. If you only want to process files containing three letters followed by four numbers and with a suffix of .hl7, the Regular expression type pattern of \w{3}\d{4}\.hl7 would be used. For a file to be processed, the pattern must match the complete filename. The default Wildcard pattern of * will match all files in the directory.

Input Order

When there are multiple files in the input directory, the order those files are processed can be defined using the **Input Order** property. This property supports ordering by filename (ascending and descending), modification time (most recent and least recent), and custom ordering.

Debatching

Incoming files can be debatched in two ways:

- **Zip File Debatch Mode**

Incoming Zip files can be unzipped automatically.

- **Debatch Mode**

Incoming files containing multiple records (such as a CSV-formatted file) can be separated by identifying the expression that either marks the start of a message or is used to separate them.

Message Properties

When the Directory Communication point reads a file from disk, it will create several properties containing information about the file. Among these are the following:

- **FullFilename**
The filename of the message.
- **BaseFilename**
The filename excluding the suffix.
- **Suffix**
The filename suffix.

These can be used in your configuration where message processing requires knowledge of the filename associated with a message.

Directory Paths

The path for the input directory should be the absolute path to the directory on the server hosting the Rhapsody engine. While the configuration dialog supports the use of forward slash (/) as well as backslash (\) in the path, the path is interpreted by the server operating system. The relevant separator should be used: backslash (\) for Windows platforms or forward slash (/) for any other platform.

UNC (Universal Naming Convention) paths are supported for accessing directories on remote servers, provided that the Rhapsody server is a Windows host and the Rhapsody engine runs as a Windows domain user. The path specification for a UNC path is as normally specified for the share:

\\Host\share\path_to_file

Output Mode

In Output mode, the Directory communication point writes the messages into a file in the defined output directory. By default, each message is written to an individual file unless the **Batch Mode** property is set to **true**.

The filename is defined using the **Base Filename** and **Suffix** properties. If these are not provided, the system default (date and time, including milliseconds) is used. No suffix is appended by default. These properties can be configured to use a message property.

Conflicting Filenames

Conflicting file names are handled by appending an incrementing number to the filename. For example, if *test.csv* exists in the output directory, the new file will be *test(1).csv*. Alternatively, the communication point can be configured to append the current date and time to the filename or overwrite the existing file.

Bidirectional Modes

When the communication point is configured for Bidirectional, In->Out or Out->In modes, the parameters for both input and output traffic are displayed. When using one of the Bidirectional modes, the communication point will appear on both the input and output sides of a route.

Lesson 17 of 23

Database

The Database communication point provides support for reading from and writing to database tables. Native support is provided for:

- **Microsoft SQL Server**
- **Oracle**
- **MySQL**
- **PostgreSQL**
- **Other third-party databases for which JDBC drivers are available.**

① **The configuration and use of the Database communication point is described in greater detail in the *Rhapsody Professional* course.**

Output Mode

In Output mode, the Database communication point executes one or more queries against a database to update the database with details from a message.

Database Properties

Notes | Support Notes | Auxiliary Files

General | Connection | Management | Configuration | Scheduling | Message Tracking

Mode: Output

Properties with an asterisk (*) are required.

Property	Value	
Database	Manual Setting	
Host *		
Port		
Database Name *		
Username		
Password		
Additional Connection Properties		...
Configuration File *	Create Configuration	...
Query Timeout		
Socket Timeout	0	
Message Type	Plain Text	
Definition		...
Message Errors	Treat As Message Error	

OK Cancel Apply Help

Out->In Mode

In Out->In mode, the Database communication point can be used to update a database (similar to Output mode) and take query results and update the message (for example, performing a code conversion or extracting additional demographic details for a patient).

Input Mode

In Input mode, the Database communication point executes SQL statements to poll the database looking for new records. These records are returned as an XML formatted message. This communication point can be configured to retain a key value from the most recently retrieved record to assist in retrieving new records (to help ensure that we only retrieve a record from the database once).

The Database communication point in Input mode will repeatedly query a database to return records. The operation can be described as follows:

1. Query the database.

2. While records are found:
 1. Records returned as an XML-formatted message.
 2. Immediately query the database again.
3. Sleep for the defined polling period.
4. Repeat process to query the database.

If the input statements are not correctly configured, the Database communication point will repeatedly query the database and rapidly return a very large number of messages.

Lesson 18 of 23

Timer

The Timer communication point is used to repeatedly send the same file to a route for processing. This input-only communication point is frequently used to test route functionality.

The screenshot shows the 'Timer Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Input'. A table lists properties: 'Body Template File' with value 'admissions.xml' and 'Refresh Rate *' with value '5000'. A note at the bottom states: 'This file will be used as the body of the incoming message.'

Property	Value
Body Template File	admissions.xml
Refresh Rate *	5000

The communication point will generate a message containing the body template file as soon as it has started, with subsequent messages sent according to the **Refresh Rate** (defined in milliseconds).

Lesson 19 of 23

E-mail Client

The E-mail Client communication point is used to exchange information between Rhapsody and one or more external systems.

Input Mode

The E-mail Client communication point in Input mode reads messages in from either POP3 or IMAP email server.

The screenshot shows the 'E-mail Client Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Input'. A table lists various properties, some of which are required (marked with an asterisk). The 'Mail Server' is set to '\$(Email_Host)', 'Store Protocol' to 'IMAP', and 'IMAP Port' to '143'. Other properties like 'POP3 Port', 'Use SSL', 'SSL Protocol Mode', 'SSL Cipher Suites', 'Trusted Certificates', 'Read Timeout (ms)', 'Connection Timeout (ms)', 'User', 'Password', and 'Folder' are also listed. The 'Folder' is set to 'INBOX'. A text area at the bottom provides a description for the Mail Server property.

Property	Value
Mail Server *	\$(Email_Host)
Store Protocol *	IMAP
POP3 Port	
IMAP Port	143
Use SSL *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Trusted Certificates	
Read Timeout (ms) *	10000
Connection Timeout (ms) *	30000
User	
Password	
Folder	INBOX

The hostname/IP address of the mail server used to receive the messages.

Output Mode

The E-mail Client communication point in Output mode constructs an email using the message as either the body or an attachment. The email can be sent to one or more recipients. A typical use is to alert an administrator of specific events, such as an external system rejecting messages or messages being received by Rhapsody with invalid formatting.

The screenshot shows the 'E-mail Client Properties' dialog box with the 'Configuration' tab selected. The 'Mode' is set to 'Output'. Below the tabs, a note states: 'Properties with an asterisk (*) are required.' A table lists various properties and their values. At the bottom, there are buttons for 'OK', 'Cancel', 'Apply', and 'Help'.

Property	Value
Mail Server *	\$(Email_Host)
Protocol *	SMTP
Port	25
Use SSL *	☐
SSL Protocol Mode *	
SSL Cipher Suites *	
Trusted Certificates	
Read Timeout (ms) *	10000
Connection Timeout (ms) *	30000
User	
Password	
Sending Mode *	Normal
From *	\$(Email_From)
Reply To	
To *	\$(Email_Admin_Recipient)

Exercise - The Directory Communication Point

We previously covered an overview of the use and configuration of some of the most commonly used communication points. In this exercise we will take a deeper look at the Directory communication point, along with some of its configuration options.

Resources

This exercise makes use of the messages available in the *.\Rhapsody\Associate Files and Definitions\105 - Communication Points\Directory Communication Point* folder in the resources ZIP file available at the start of the course.

Directory and File Naming

This exercise uses a Directory communication point in Bidirectional mode.

Exercise Tasks

1. 1

Create a working folder in the **Workspace** for this exercise, followed by a new route inside this folder.

2. 2

Create a Directory communication point and configure as follows:

- **Connection Mode:** Bidirectional
- **Input Directory:** *c:\Rhapsody\Associate Exercises\REN-105\Directory Communication Point\Input*
- **Output Directory:** *c:\Rhapsody\Associate Exercises\REN-105\Directory Communication Point\Output*

3. 3

Add the Directory communication point to both the Input and Output panels of the route.

4. 4

Connect the Input and Output communication points, then check in the route and start all the components.

5. 5

Place copies of the ten test messages found in the resource directory into the input folder. They will be detected by the communication point and passed to the route. Following processing, the files are then written to the output folder.

6. 6

Verify that the files are in the output folder. Since we have not defined a filename structure for output messages, the default timestamp naming convention will be used.

It is possible for two or more messages to simultaneously arrive at the output directory. When this happens, an incrementing number in parentheses will be appended to the end of the filename to avoid name conflicts.

Using Rhapsody Variables

We can use **Rhapsody Variables** to simplify the repeated use of values and the migration of a configuration between different engine environments. They are commonly used in the configuration of a Directory communication point's input and/or output pathway. A variable is specified in two ways:

- Manually
Type the reference to the variable using the syntax `$(<variable_name>)`.
- Automatically
Select the **Variables** icon then select the variable's name from the list.

We created a variable named `RhapsodyAssociate_TestDirectory` in module 104 with a value of `c:\Rhapsody\Associate Exercises` (or similar). We will reuse this variable in this exercise to create several directories. The variable can be combined with one or more sub-directories as shown below:

- `$(RhapsodyAssociate_TestDir)\REN-105\Directory Communication Point\Input`
- `$(RhapsodyAssociate_TestDir)\REN-105\Directory Communication Point\Output`

Using variables in this way helps ensure a consistent location for files and reduces the chance of typing errors when entering the path.

Exercise Tasks

1. 1

Modify the configuration of the Input and Output Directory properties to match those shown in the above example.

- Check in the change.
- Place copies of the test messages from the Input Throttling Behavior exercise into the Input directory.

2. 2

Confirm that the operation of the communication point matches that seen in the earlier exercise.

Output File Naming

The Directory communication point in any of its output modes (Output, Bidirectional, In->Out, Out->In) uses the **Base Filename** and **Suffix** properties to set the name given

to files as they are written to the output directory. There are several ways in which these properties could be specified:

- By providing a string to use as the filename and suffix.
 - If multiple files are written to the same directory, an incrementing number will be appended to the filename.
For example LabResult, LabResult(1), LabResult(2).
 - A path may be appended to the filename. For example, if *messages\output\LabResult* is specified, the output file will be named LabResult and saved in the *\messages\output* directory under the **OutputDirectory**. If this directory does not exist, it will automatically be created.
- Using a **Rhapsody Variable**. This can be selected from a drop-down list or by direct entry, using the syntax \$(<variable_name>).
- Using a **Message Property**. This is typed in using the syntax \$<message_property_name>. A Message property can be hard coded by a filter or extracted from a field within a message (for example, the name of a hospital or laboratory). Properties can also be automatically created by a communication point or filter. View the **Help** page to see details of published properties associated with the Directory communication point.

Exercise Tasks

Modify the configuration of the **Base Filename** and **Suffix** properties for the Directory communication point by completing the following actions:

1. 1

Create a new Rhapsody variable using a name and value of your own choosing.

2. 2

Specify this variable as the base filename for output files and check in all changes.

3. 3

Place copies of the test messages into the Input folder.

4. 4

Confirm that the names of messages saved to the Output directory match the specified Rhapsody variable and that an incrementing number has been added to each file.

5. 5

Modify the configuration of the **Base Filename** and **Suffix** properties to be \$BaseFilename and \$Suffix respectively.

- Check in the change.
- Place copies of the exercise test messages into the Input directory.

6. 6

Confirm that the names of messages saved to the Output directory are the same as those of the original input messages.

Input Pattern Masking

File selection from an Input directory can be controlled using a pattern which defines the set of file names to be included or excluded by the communication point. This option is set in the **Pattern** and **Exclude Pattern** sections of the **Configuration** tab of the Directory communication point's **Properties** dialog.

The **Pattern** specification requires a pattern that matches the complete filename. When using the wildcard pattern type, the * character is used to represent the wildcard text. The pattern may be built using a combination of text and the wildcard to define the portions of the file names that are not important.

For example:

- *
- Defines a match for any filename
- *.txt
- Defines all filenames ending in .txt
- Input*
- Defines all filenames beginning with Input

The * character is the only wildcard character that can be used.

Exclude pattern specification

The **Exclude Pattern** defines a pattern for files not to read. A filename must match the complete pattern to not be read. For example;

- *.txt will not input files ending with the term .txt

Exercise Tasks

1. 1

Modify the **Exclude pattern** of the Directory communication point to exclude all test filenames that include the string 't' in their name. Hint - you will need to use wildcards.

2. 2

Ensure that the route and components are checked in and running.

3. 3

Copy the four test files to the Input directory.

4. 4

Confirm that the operation of the communication point matches the expected behavior.

What you should see

After a few seconds, all files should have been removed from the Input directory and passed on to the route, except for those with the exclude pattern in their names (message.temp and message.txt).

Message Batching/De-batching

It is common for file-based messaging to include a batch of messages combined into a single file. The Directory communication point supports extracting individual messages from this file using:

- EITHER a marker for the start of each message.
- OR a separator at the end of each message.

The *PatientDetails.csv* test file, contains dummy patient information in CSV format. The end of each line in this file is marked by the Linefeed character (represented as \n).

Exercise Tasks

Create a new route with a single, bidirectional Directory communication point on the Input and Output sides of the route editing canvas with a connector between them.

Configure the communication point with the following settings:

- **De-batch Mode:** Selected
- **Separator Type:** Separates messages
- **Message Separator Expression:** \n

Check in and start all components. Place a copy of the test file in the Input folder. Verify the results in the Management Console and in the communication point's Output folder.

What you should see

The *PatientDetails.csv* file includes six patient records plus a header row. The output from this route will therefore be seven files, each representing one of the rows in the original message.

Exercise - The TCP Client Communication Point

In this exercise we will look at the TCP Client communication point and investigate how it functions using various connection modes and server behaviors.

Resources

This exercise makes use of the messages and message definitions available in the in the `.\Rhapsody\Associate Files and Definitions\105 - Communication Points\TCP Communication Points` folder in the resources ZIP file available at the start of the course.

Setup

This exercise uses an example configuration to provide something for the TCP Client you will configure to connect to. Start by importing *TCP Client Communication Points.rlc* into your Rhapsody engine. The process for uploading an RLC file to a Rhapsody engine was discussed in Saving and Reloading a Configuration lesson of the "Managing Your Configuration" module.

① The routes and communication points in the RLC must be running while you are working on this exercise.

Remote host and remote port properties

Along with the routes and communication points, the provided RLC file includes several Rhapsody variables. These variables define the hostname and port for each of the test services you will be connecting to in the exercise.

This exercise uses the following Rhapsody variables added to your Rhapsody engine when the *TCP Client Communication Points.rlc* file was imported.

Variable Name	Value
TCP_Remote_Host	localhost
TCP_Remote_Port	10501

Output Mode

In the first part of this exercise, you will construct an output-only TCP Client that sends messages to a server.

1. 1

Create a working route for this exercise that includes a bidirectional Directory communication point that utilizes suitable file locations on your computer.

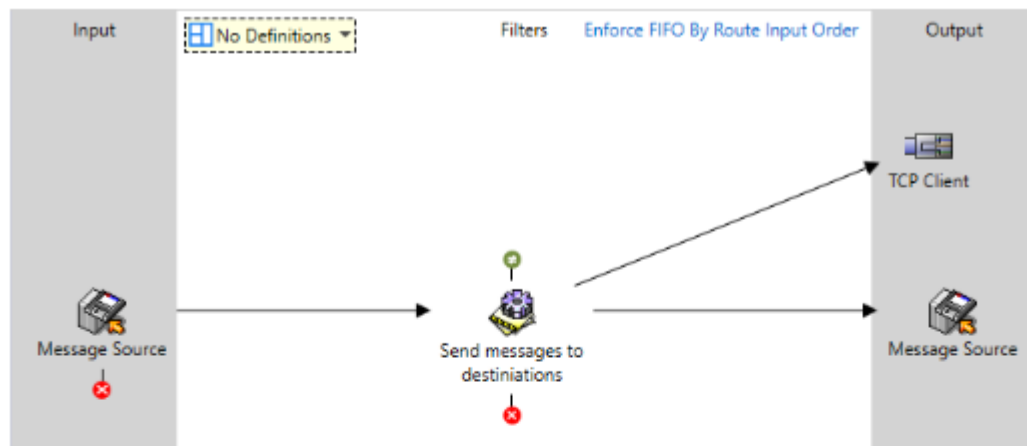
2. 2

Create a TCP Client communication point and configure as follows:

- **Connection Mode:** Output
- **Remote Host:** \$(TCP_Remote_Host)
- **Remote Port:** \$(TCP_Remote_Port)
- All other parameters can be left at their default values.

3. 3

Add the communication point to the output of the following route and connect it as shown below:



- 4 Check in and start all components.
- 5 Ensure the TCP Client communication point starts without error. Note that this step will fail if the three Resources routes installed in the Rhapsody engine for this module are not checked in and running.
- 6 Copy the *HL7v2.4_ADTA01.edi* test message to the Directory communication point's Input folder.

What you should see

1. The test message leaves the Directory communication point's input directory and a copy is placed in its output directory.
2. The test message is also sent by the TCP Client communication point, but no response is returned.

Model Answer

The test message sent by the TCP Client communication point does not return a response because the message was received by the TCP Server communication point in the downloaded route, then discarded to a Sink communication point.

The following screenshot shows a filtered view of the current communication points, with the working and downloaded ones highlighted.

COMMUNICATION POINT	TYPE	STATUS	ACTION	CONNECT	IN QUEUE	OUT QUEUE	FAILED QU	RECEIVED	SENT	FAILED	IDLE TIME
Exercise											
REN-105											
Communication Properties...											
TCP Client Communication Point		Running	Stop	1	0	0	0	1	1	0	26 secs
Message Source	Direct...	Running	Stop	1	0	0	0	0	1	0	26 secs
TCP Client	TCP C...	Running	Stop	1	0	0	0	0	1	0	26 secs
Rhapsody Academy											
Rh...											
Rh...											
TCP Client Com...											
1. Connectio...											
Sink	Sink	Running	Stop	1	0	0	0	0	1	0	26 secs
TCP No R...	TCP S...	Running	Stop	1	0	0	0	1	0	0	26 secs
2. Out-In Mo...											
3. Retry Bloc...											

Out->In Mode

When the communication point is configured it sends a message, then blocks output until a response has been received. This mode requires that the communication point be configured on the Input and the Output sides of a route (normally the same route).

This exercise uses the following Rhapsody variables added to your Rhapsody engine when the *REN-105 Communication Points.rlc* file was imported.

Exercise Tasks

This exercise establishes

Variable Name	Value
TCP_Remote_HL7_Host	localhost
TCP_Remote_HL7_Port	10502

a

synchronous connection using a TCP Client communication point in Out->In mode to a remote server and waits until a response has been received from it.

- 1 Create the following route, or adapt it from the previous one:



- 2 Configure the TCP Client communication point with the following parameters:

- **Connection Mode:** Out->In
- **Remote Host:** `$(TCP_Remote_HL7_Host)`
- **Remote Port:** `$(TCP_Remote_HL7_Port)`
- **Strip Wrapping:** Selected

- 3 Check in and start all components.

- 4 Ensure the TCP Client communication point starts without error. Note that this step will fail if the three Resources routes installed in the Rhapsody engine for this module are not checked in and running.

- 5 Copy the `HL7v2.4_ADTA01.edi` test message into the Directory communication point's Input folder.

What you should see

The test message is picked up from the Directory communication point's Input directory. Shortly afterwards, a response message is copied to the Output directory. The remaining outcomes for this exercise are viewed in the Rhapsody Management Console.

1. From the **Monitoring** menu, select **Communication Points**. Locate and select the **TCP Client** communication point created for this exercise.
2. From the **Details** screen, select the **View Output Archive** link. Select the most recently added message.
3. Select the **TCP Client** communication point's entry in the message's **Event Tree**.
4. Click the **Response received by message tracking** link to split the screen to show the Output Message **Properties** and **Body** panels on the left of the screen, and the Response Message **Properties** and **Body** panels on the right.

Model Answer

When a communication point in Out->In mode sends a message, it must wait for a matching response before it can send the next message. This is shown in the following screenshot taken from the TCP Client communication point's Output archive. The output message is the HL7v2.4_ADTA01.edi test message copied into the Directory communication point's Input directory. The response message was generated remotely by a route installed from the downloaded RLC file and returned to the TCP Client communication point as an acknowledgement.

The following screen shows a message from the TCP Client's output archive in split view, with the output and response messages on the left and right of the screen respectively. This view is created by selecting the **Response received by message tracking** link.

Message View 1.8597445376.185262

Displaying message 1 of 3 from the result set named 'TCP Client Output Archive' Output archive for TCP Client communication point

Reinject Redirect Copy Redirect Edited Copy Resend

Message Events

Always Show Outputs Last refreshed at 12:14

COMPONENT	TIMESTAMP	DESCRIPTION
Message Source	Oct 11, 2022 12:14:38.124	Received by communication point
Out-In	Oct 11, 2022 12:14:38.124	Placed on route
TCP Client	Oct 11, 2022 12:14:38.124	Delivered to communication point
	Oct 11, 2022 12:14:38.125	Sent by communication point, total proces...
	Oct 11, 2022 12:14:38.125	Response received by message tracking

Response message received Select to display response message

Message Properties

Output Message

Property Name	Property Value
BaseFilename	HL7v2.4_ADTA01
directory:BodySize	487
FileCreationTime	1599723251000
FullFilename	HL7v2.4_ADTA01.edi
InputTime	1665443678124
rhapsody:InputCommunicationPoint	Message Source
rhapsody:InputCommunicationPoin...	110
Suffix	.edi

Showing 8 of 8

Response Message

Property Name	Property Value
BaseFilename	HL7v2.4_ADTA01
directory:BodySize	487
FileCreationTime	1599723251000
FullFilename	HL7v2.4_ADTA01.edi
InputTime	1665443678126
rhapsody:InputCommunicationPoint	Message Source
rhapsody:InputCommunicationPoin...	110
Suffix	.edi

Showing 8 of 8

Message Body

Tree View Flat Wire

Output Message Show Whitespace Encoding: US-ASCII Size: 487 bytes

```
MSH|^~\&|Rhapsody|Rhapsody^PIT^ODL|CDR|Test|20090917105857||ADT^A|EVN||20090917105846|PDI||84568-4564^MRN^MRN^104532R^MRN^MHR||Cardinal^John^Q||19PD1||Bough Family Clinic^1101|0x10-000998^MOOD^Brandon^R^D^NK1|0|Jargon^Carol|Sister^Sister||408|455-2112|PV1||I|Q5-H1^Bd003B^F1003|I||Qx10-000524^Jose^A|Alerbento^A^D|PV2||ORTON^Congestive heart failure|
```

Download

Response Message

Show Whitespace Encoding: US-ASCII Size: 99 bytes

```
MSH|^~\&|CDR|Test|Rhapsody|Rhapsody^PIT^ODL|20200702155347419+120MSA|AA|3CA8141|
```

Download

Details for response message

Retry Blocking

The TCP Client communication point provides support to manage the behavior when a network connection to the remote host fails. The retry behavior is configured on the **Connection** tab (**Connection Retries** panel) of the communication point's **Configuration** dialog. This exercise illustrates the sequence of events if the remote server does not respond to a message in a timely manner.

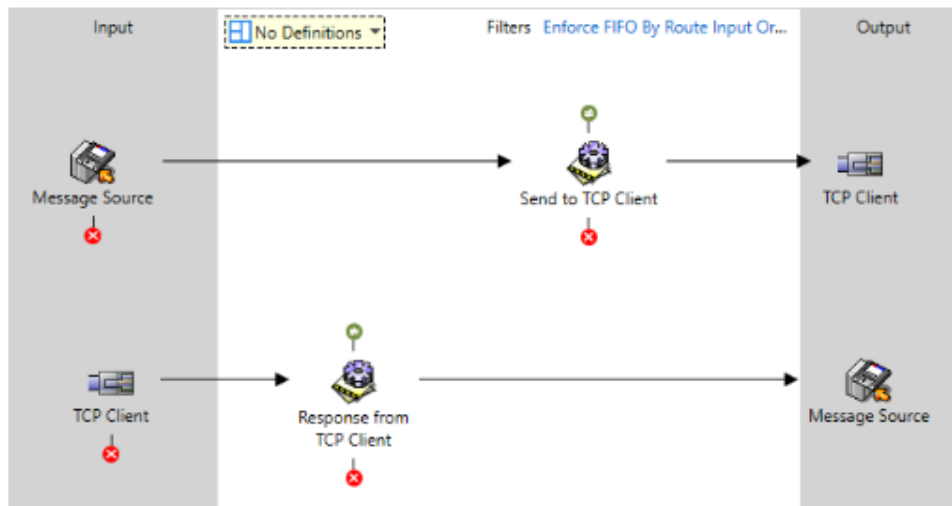
The **Properties** dialog for a communication point in Out->In mode provides support for situations where a response is not received from the server. This allows recovery when the message is sent but no response is received within the period defined by the **Response Timeout** and ensures that the communication point does not wait for the response indefinitely and blocks output of later messages.

This exercise uses the following Rhapsody variables added to your Rhapsody engine when the *REN-105 Communication Points.rlc* file was imported.

Variable Name	Value
TCP_Blocking_Host	localhost
TCP_Blocking_Port	10503

Tasks

- 1 Either reuse or create a copy of the route used in the previous exercise:



- 2 Reconfigure the TCP Client communication point with the following parameters:
 - **Connection Mode:** Out->In
 - **Remote Host:** `$(TCP_Blocking_Host)`
 - **Remote Port:** `$(TCP_Blocking_Port)`
- 3 On the communication point's **Connection** tab, navigate to the bottom of the screen and specify the following **Out->In Properties**:
 - **Response Timeout (ms):** `10000`
 - **Suspend message timeout once response is received:** <not selected>
 - **Retry Count:** `1`
 - **Fail Action:** Treat as error
- 4 Ensure all components are checked in and running. Note that the TCP Client communication point will fail to start if the three Resources routes installed in the Rhapsody engine for this module are not checked in and running.
- 5 Copy the `HL7v2.4_ADTA01.edi` test message to the Directory communication point's Input folder.

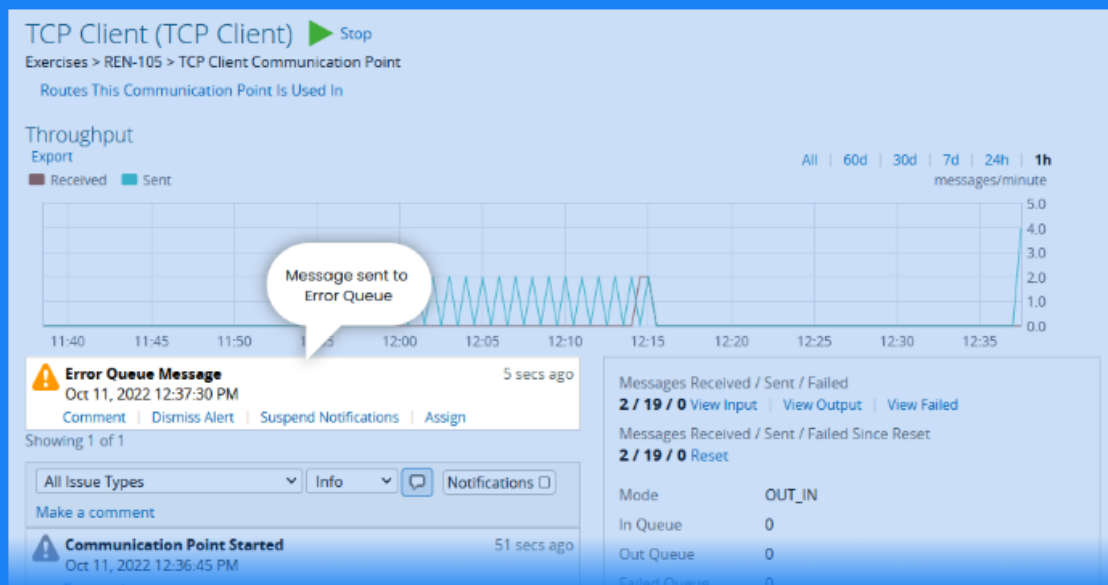
What you should see

The test message is picked up from the Input directory. However, no response message appears in the Output directory. The remaining outcomes for this exercise can be viewed in the Management Console.

1. From the **Monitoring** menu, select **Communication Points**.
 1. Select the **TCP Client** communication point configured for this exercise.
 2. Note the alert stating that a message has been sent to the Error Queue.
 3. Note the notification explaining the reason for this action.
2. From the **Monitoring** menu, select the **Error Queue**.
 1. Select the most recent message added to this queue from the TCP Client communication point.
 2. On the message's **Details** screen, note that the message was re-routed by message tracking to the Error Queue.

Model Answer

On the TCP Client communication point's Details screen, an alert is displayed showing that a message has been delivered to the Error Queue. The communication point's most recent log entry identifies the reason for this action.



Message view at output

On the Details screen for a message selected from the Error Queue, and in its **Message Events** panel, the message's initial sending event is followed by the subsequent resend 10 seconds later. This is because the Response Timeout requires that the response be received within 10 seconds. If this doesn't happen, the message is resent.

Eventually the resend fails and the message is routed to the Error Queue.

Message View 1.8597948288.189161

◀ Displaying message 1 of 1 from the result set named 'Error Queue' Back to Result Set ▶

Reinject Redirect Reprocess Edit Delete

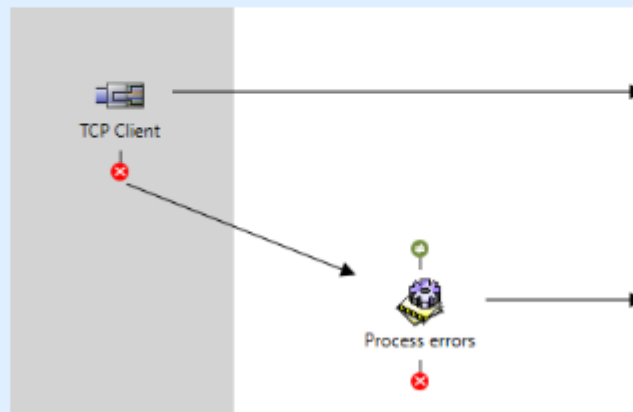
Message Events ▾

Always Show Outputs ☒

Last refreshed at 12:49

COMPONENT	TIMESTAMP	DESCRIPTION	
Retry Blocking	Oct 11, 2022 12:37:10.068	Placed on route	
Send to TCP Client	Oct 11, 2022 12:37:10.073	Filtering succeeded; took 0.004 ms	
TCP Client	Oct 11, 2022 12:37:10.073	Delivered to communication point	
	Oct 11, 2022 12:37:10.075	Sent by communication point, total proc...	

① If we add configuration that connects to the error connector on the TCP Client communication point, the failed message will be sent there instead of the Error Queue. This lets us manage messages that fail to be sent using Rhapsody configuration.



Lesson 22 of 23

Exercise - The TCP Server Communication Point

The TCP Server communication point listens for connections on the port specified by the **Local Port** property. It can be bound to a specific interface by supplying the hostname/IP address bound to that interface in the **Local Address** property.

This exercise will provide an in-depth look at the TCP Server communication point and its ability to connect to an external system, which in this case will be EDI Explorer.

Resources

This exercise makes use of the messages and definitions available in the *.\Associate Files and Definitions\105 - Communication Points\TCP Communication Points* folder in the resources ZIP file available at the start of the course.

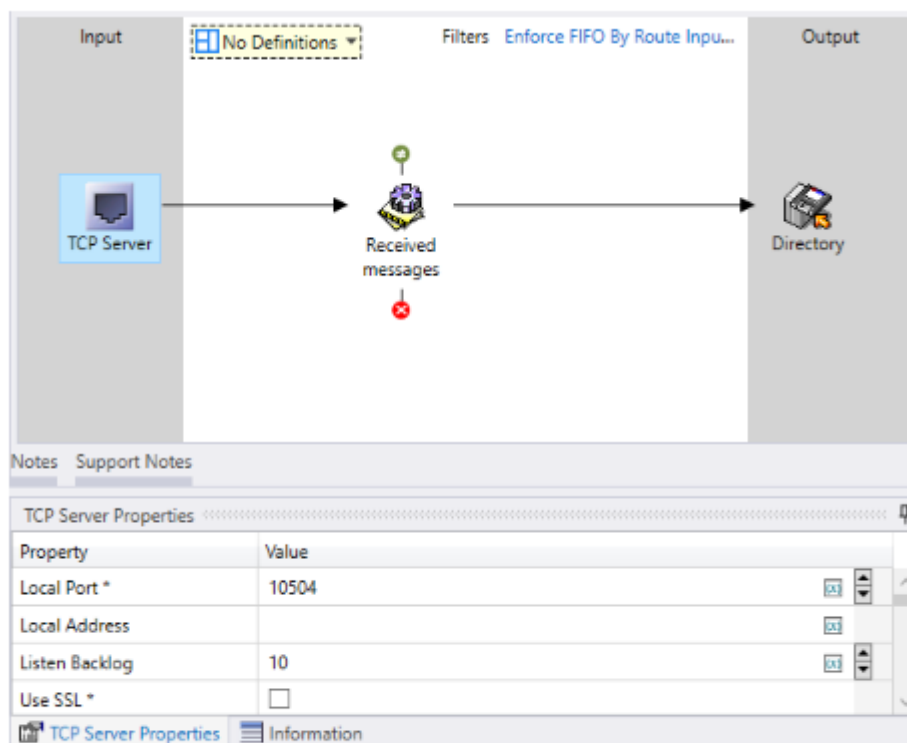
Setup

Create the route shown below. The TCP Server communication point needs to be configured in **Input** mode and its **Local Port** setting is 10504 (or some other available number). This is the port number across which the communication point will listen for connections.

It is best practice that the TCP Server's **Local Port** property be configured using a Rhapsody variable.

The Directory communication point needs to be configured in Output mode and added to the output of the route. The **Output Directory**, **Base Filename**, and **Suffix** options

should be set to values of your choosing that will make it easy to review the received messages on disk.



TCP server Route layout including TCP Server communication point settings.

Check in and start the route and its two communication points.

Configure the sending system

This exercise uses EDI Explorer configured to send messages to the configuration we have created in Rhapsody. EDI Explorer will be configured to send messages to the local machine on the port number configured in the TCP Server communication point.

1. 1

Open EDI Explorer.

2. 2

From EDI Explorer's **File** menu, select **Open Definition** and then select the *Academy-HL7v2.4-ADT.s3d* message definition. The definition's name should appear in the EDI Explorer's status bar.

3. 3

Ensure **TCP Client** is selected in the drop-down on the EDI Explorer toolbar.

4. 4

From the **Communications** menu, select **Communication Setup**. Confirm or enter the following parameters:

- **Host Address:** localhost
- **Port Number:** 10504
- All other parameters can be left at their default values. Select **OK** to close this dialog.

5. 5

From the **Tools** menu, select **Options** followed by the **Acknowledgements** tab. Ensure that the **Enable Acknowledgements** checkbox is not selected. Select **OK** to close this dialog.

6. 6

From the **Communications** menu, select **Connect**. The label **Connected** should appear in EDI Explorer's Status Bar.

Send a test message

1. 1

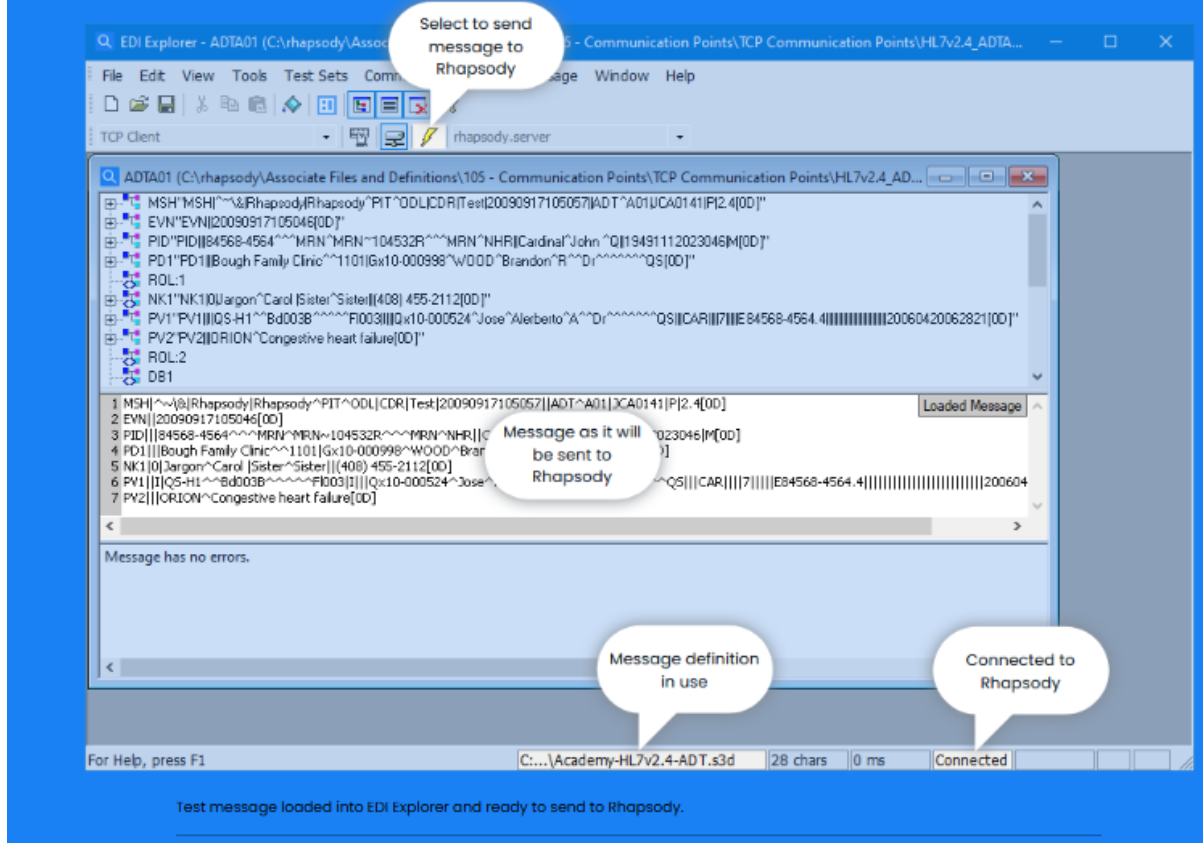
From EDI Explorer's **File** menu, select **Import Flatwire Message From File**. Select the *HL7v2.4_ADTA01.edi* message. The HL7 file will appear in EDI Explorer.

2. 2

From the **Communications** menu select **Start Sending**. This will send the message to Rhapsody.

Model Answer

Inspect the message saved to disk by the Directory communication point. It should appear identical to the message you loaded into the EDI Explorer.



Using the EDI Explorer as a testing tool

This exercise illustrates a valuable use of the EDI Explorer application - as a testing tool for TCP Client and Server connections. It can also be used to validate test messages against the loaded message definition.

An additional feature of EDI Explorer is its ability to automatically generate an acknowledgement for every message it receives, and to return it back into Rhapsody. This option will be explored in a later module.